ZERO-TO-PRO MACHINE LEARNING TUTORIAL

DBSCAN & Optuna

Clustering by density and smart hyperparameter tuning — explained from basics to advanced with diagrams, math, worked examples and code.

● **What is inside**

Part A — DBSCAN: why K-Means is not enough, ε and minPts, core / border / noise points, density-connectivity, the algorithm, scikit-learn code, tuning, limitations, interview questions.

Part B — Optuna: Grid vs Random vs Bayesian search, TPE, the five key terms, a complete worked study, samplers, the five visualizations, define-by-run, pruning, storage, parallelism, interview questions.

Built from two lecture transcripts · every topic in the transcripts is covered · dark theme for eye comfort

Contents

PART A — DBSCAN: clustering by density 3

A1 · Why do we need DBSCAN? (K-Means has three flaws) . . . 3
A2 · What is density-based clustering? . . . 4
A3 · The two hyperparameters: ε (epsilon) and minPts . . . 5
A4 · The three types of points: core, border, noise . . . 6
A5 · Density-connected points (how clusters grow) . . . 8
A6 · The DBSCAN algorithm, step by step . . . 10
A7 · DBSCAN in scikit-learn (the examples from the lecture) . . . 11
A8 · Pro level: choosing ε and minPts, scaling and metrics . . . 13
A9 · Advantages and limitations . . . 15
A10 · Build solid intuition: the interactive visualization tool . . . 16
A11 · Cheat-sheet and interview questions . . . 17

PART B — Optuna: smart hyperparameter tuning 19

B1 · Why Optuna? The problem with Grid Search and Random Search . . . 19
B2 · Bayesian optimization from scratch . . . 20
B3 · TPE — the sampler Optuna uses by default . . . 23
B4 · The five key terms (learn these before reading Optuna code) . . . 24
B5 · Hands-on: your first Optuna study . . . 25
B6 · Samplers: Bayesian, Random and Grid with the same objective . . . 28
B7 · Optuna's visualizations — see what happened . . . 29
B8 · Define-by-Run: dynamic search spaces (the lecturer's favourite) . . . 32
B9 · Pro level: features that make Optuna production-ready . . . 34
B10 · Best practices, cheat-sheet and interview questions . . . 36

References and further reading 38

✔ How to read this tutorial

Each part goes **basic → advanced**. Blue ● boxes = key ideas, purple ∑ boxes = the math (skip on a first read if you like), orange ▲ boxes = warnings and pro tips, yellow ▶ boxes = notes about the original video, green ✔ boxes = practical tips. Code blocks are runnable.

Prerequisites: basic Python and NumPy, a feel for what clustering and hyperparameters are, and (for Part B) scikit-learn's `cross_val_score` and Random Forests. Everything else is explained from scratch.

Dark theme: every page uses a dark background and light text to be comfortable for the eyes.

PART A — DBSCAN: clustering by density

What you will be able to do after this part. Explain in plain words why K-Means is not always enough, define ε, minPts, core / border / noise points and density-connectivity, run DBSCAN by hand on a toy dataset, implement it from scratch, use it in scikit-learn, choose its hyperparameters like a professional, and answer the typical interview questions about it.

● How this part is organised (basic → pro)

A1–A2 Why we need a new clustering algorithm and what "density-based" means. **A3–A5** The four building blocks: ε and minPts, point types, density-connectivity. **A6** The algorithm step by step, plus a from-scratch implementation. **A7** scikit-learn in practice with the small examples from the lecture. **A8** Pro level: choosing ε and minPts, scaling, distance metrics, complexity. **A9** Advantages and limitations, and what to use instead. **A10** The interactive visualization tool and practice plan. **A11** Cheat-sheet and interview questions.

▶ About the source

This tutorial is built from a lecture transcript ("Hi guys, my name is Nitish…"). The lecture promises three things: geometric intuition, a small Python example, and an interactive visualization tool. All three are covered here, with the missing pictures redrawn and the math added. The lecture also stresses that DBSCAN is asked very often in interviews, so A11 is worth your time.

A1 · Why do we need DBSCAN? (K-Means has three flaws)

You already know K-Means: choose k, drop k centroids, assign each point to its nearest centroid, move every centroid to the mean of its points, repeat. Formally it minimises the within-cluster sum of squares:

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid (mean) of cluster C_k. K-Means is a good algorithm, but this very definition — *distance to a mean, with a fixed k* — creates three weaknesses.

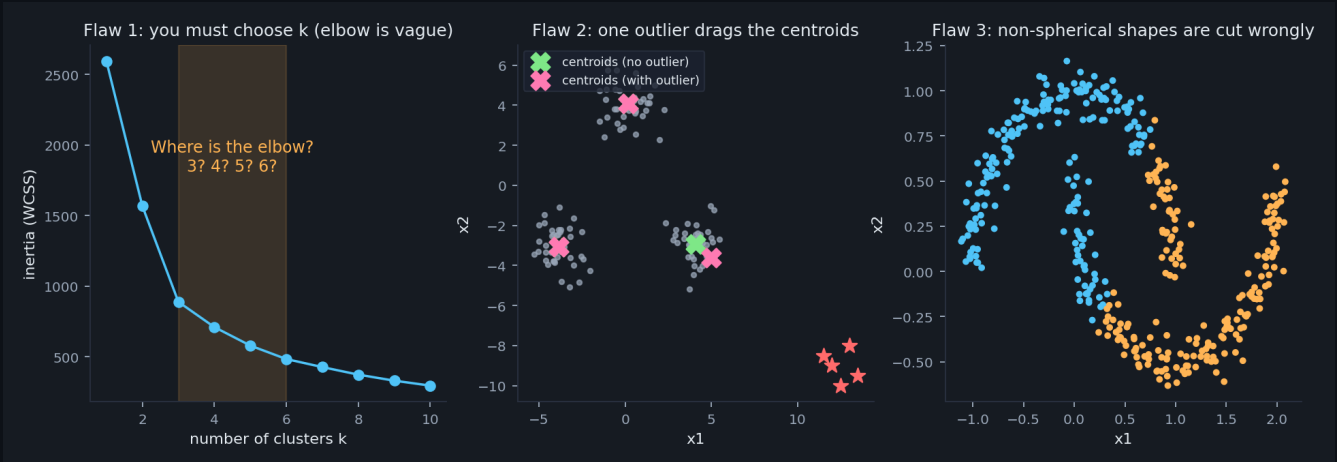


Figure A1 — The three flaws of K-Means. Left: the elbow curve has no clear bend. Middle: five far-away outliers pull the right-hand centroid (pink) away from where it should be (green). Right: K-Means cuts two interleaved half-moons with a straight line.

Flaw 1 — you must tell it how many clusters to make

Before clustering you must supply *k*. With 10-dimensional data you cannot even plot the data to look. The usual workaround is the **elbow method** (plot inertia against k and look for the bend), but as the left panel shows, the elbow is often ambiguous — is it 3, 4, 5 or 6? That makes K-Means awkward to use on real data.

Flaw 2 — it is very sensitive to outliers

K-Means keeps computing distances from every centroid to every point, and each centroid is a *mean*. A mean is dragged by extreme values, so a single outlier (or a few) can shake the centroids away from the centres of their true clusters, as in the middle panel.

Flaw 3 — it only likes round (spherical) clusters

K-Means is **centroid-based**: its whole mechanism revolves around centroids. A centroid represents a ball-shaped blob well, so when you feed it weirdly shaped data (moons, rings, spirals, snakes) it "starts crying" — it carves the space into convex Voronoi cells and clusters the data in a completely wrong way, as in the right panel.

> **● Conclusion**
> K-Means works in some scenarios but fails in many others. That gap created the need for a new type of clustering algorithm — one that needs no k, tolerates outliers and respects shape. That algorithm is **DBSCAN**.

A2 · What is density-based clustering?

K-Means is called **centroid-based** because all of its logic (create centroids, measure distances, move points, form clusters) comes from centroids. DBSCAN follows a completely different style called **density-based clustering**: it clusters data using the *density of data points*.

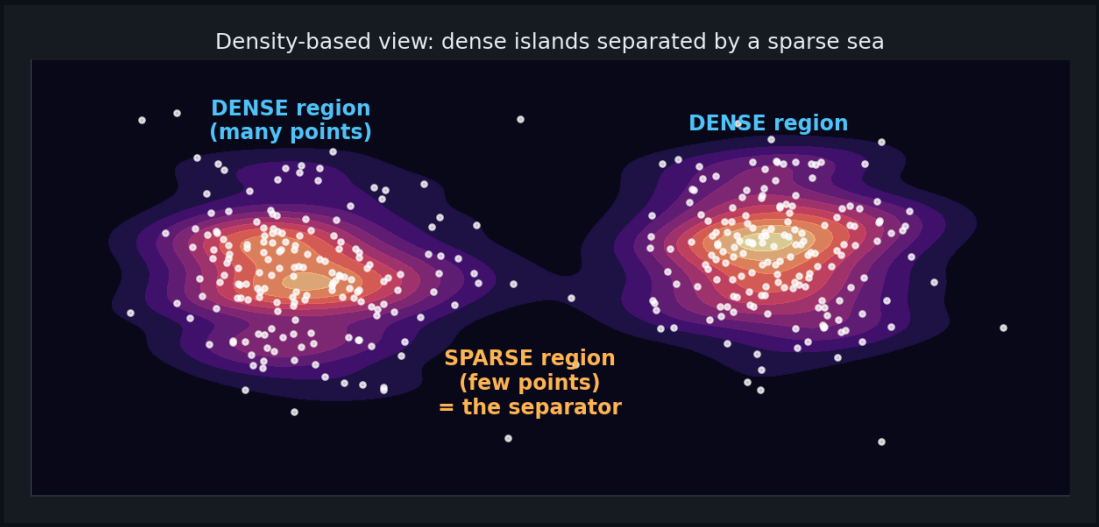


Figure A2 — Brighter colours = more points per unit area. A density-based algorithm sees two dense islands separated by a sparse region and uses the sparse region as the boundary between clusters.

Everyday analogy. Look at a country at night from an aeroplane. Cities are bright clusters of lights, the countryside between them is dark. You would never say "there are exactly k=5 centroids"; you would simply say "bright blobs separated by dark gaps" — and a lone farmhouse light in the desert is not a city, it is noise. That is exactly how DBSCAN looks at data.

The name decoded

> ● **DBSCAN = Density-Based Spatial Clustering of Applications with Noise**
>
> **Density-Based** — clusters are dense regions. **Spatial** — it works on points in a space with a distance. **Clustering** — it groups points. **of Applications** — meant for real data. **with Noise** — it can explicitly say “this point belongs to no cluster”. By the end of this part each word of the name will make sense.

Another well-known density-based algorithm is **OPTICS** (it orders points by reachability so that clusters of different densities can be extracted; see A9). Its modern cousin is HDBSCAN.

A3 · The two hyperparameters: ε (epsilon) and minPts

To talk about “dense” and “sparse” we need numbers. DBSCAN has exactly two hyperparameters — and, as the lecture says, they are the only two you must learn to tune.

Symbol	scikit-learn name	Meaning
ε (epsilon)	eps	Radius of the circle (sphere in higher dimensions) drawn around a point. The unit is whatever your data uses — 1 cm, 1 mm or 1 km, it does not matter as long as you are consistent.
minPts	`min_samples`	The minimum number of points that must be inside that circle for the area to be called *dense*. If the count is ≥ minPts it is dense, otherwise sparse.

The circle around a point p is called its **ε-neighbourhood**. In maths:

$$N_\varepsilon(p) = \{\, q \in D \;:\; \mathrm{dist}(p,q) \le \varepsilon \,\}$$

with the usual Euclidean distance between two points in d dimensions:

$$\mathrm{dist}(p,q) = \sqrt{\sum_{j=1}^{d} (p_j - q_j)^2}$$

Worked example from the lecture (ε = 1, minPts = 3)

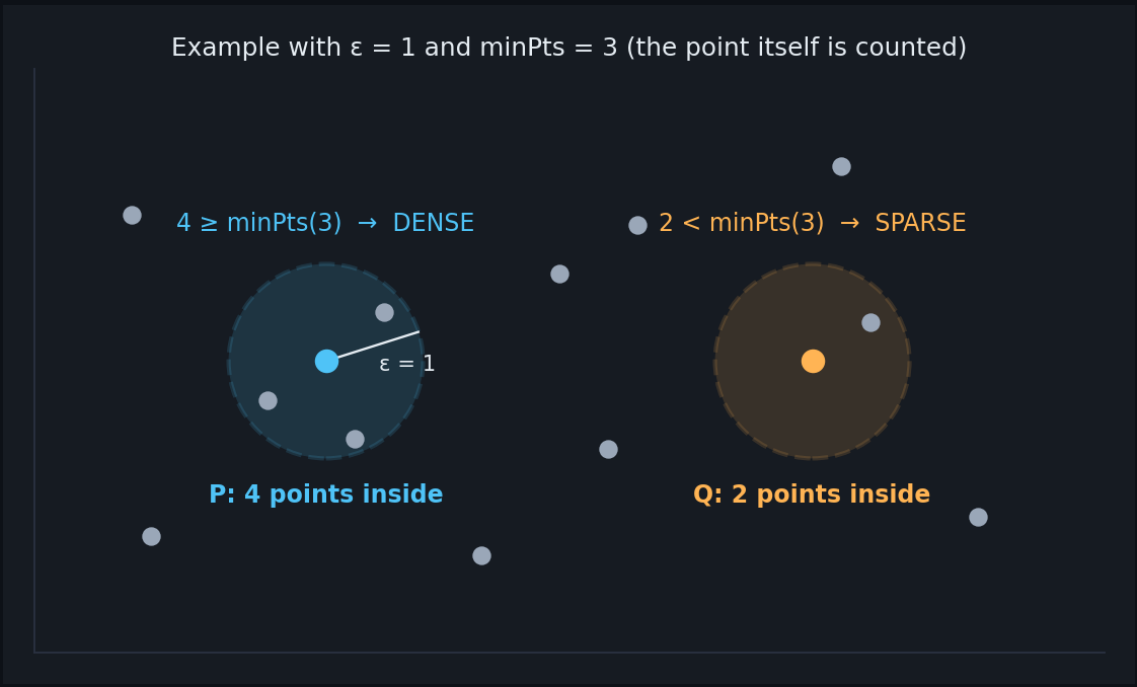


Figure A3 — Drawing the ε-circle around P captures 4 points (≥ 3) so the area is dense; around Q only 2 points (< 3) so it is sparse.

Take point P, draw a circle of radius ε = 1 and count: 1, 2, 3, 4 points → 4 ≥ 3, so the density around P is **dense**. Take point Q, draw its circle and count: 2 points → 2 < 3, so the density around Q is **sparse**. Repeating this for any point tells you the density around it. **Note:** the point itself is counted (this is also how scikit-learn counts `min_samples`).

> **✔ Intuition for the two knobs**
> **Bigger ε** → bigger circles → more points per circle → more areas are dense and neighbouring clusters may merge. **Bigger minPts** → a stricter definition of "dense" → more points become noise and clusters may split or vanish. You will see both effects in A7 and A8.

A4 · The three types of points: core, border, noise

Every point in a DBSCAN run ends up in exactly one of three roles. The concepts are simple once ε and minPts are clear.

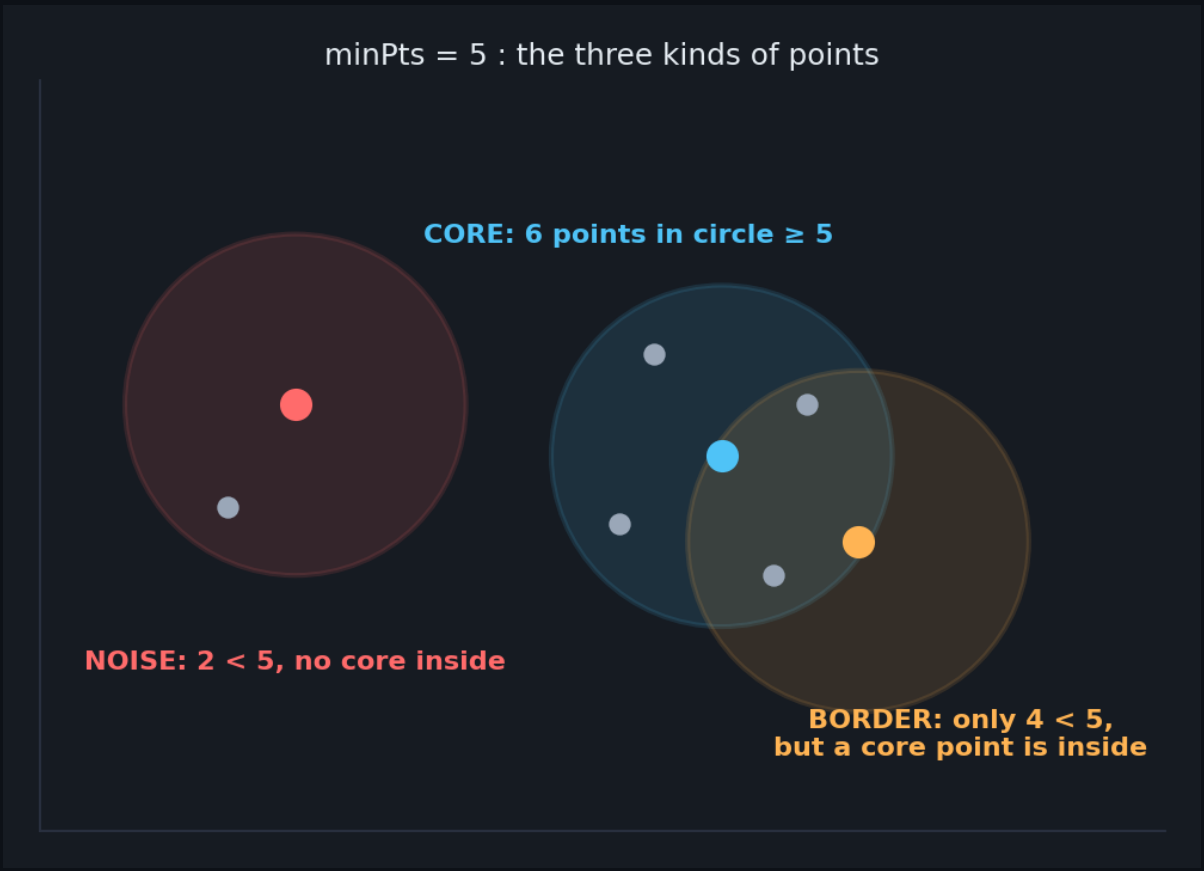


Figure A4 — The same ε = 1 circle, minPts = 5. The core point has 6 points in its circle (itself and the border point included). The border point has only 4 but contains the core point. The noise point has 2 and no core point nearby.

1 · Core point

A core point has **at least minPts points in its ε-neighbourhood** (itself included). Formally:

$$p \text{ is core} \iff |N_\varepsilon(p)| \geq \text{minPts}$$

In the lecture's picture minPts = 5: draw the red ε-circle around the chosen point, count 1, 2, 3, 4, 5 — equal to minPts (not more, but equal is enough), so it is a core point.

2 · Border point

A border point satisfies **two conditions**: (i) it has *fewer* than minPts points in its ε-neighbourhood (so it is not core), but (ii) it has **at least one core point** inside that neighbourhood. In the picture: count 1, 2, 3 → less than 5, so not core; but a core point sits inside its circle, so it is a border point.

$$p \text{ is border} \iff |N_\varepsilon(p)| < \text{minPts} \quad \wedge \quad \exists\, q \in N_\varepsilon(p) :\ q \text{ is core}$$

3 · Noise point

Anything that is neither core nor border is **noise** (also called an outlier). In the picture: count 1, 2 → less than 5, so not core; and no core point lies inside its circle, so not border either. Noise is what is left. scikit-learn labels it `-1`.

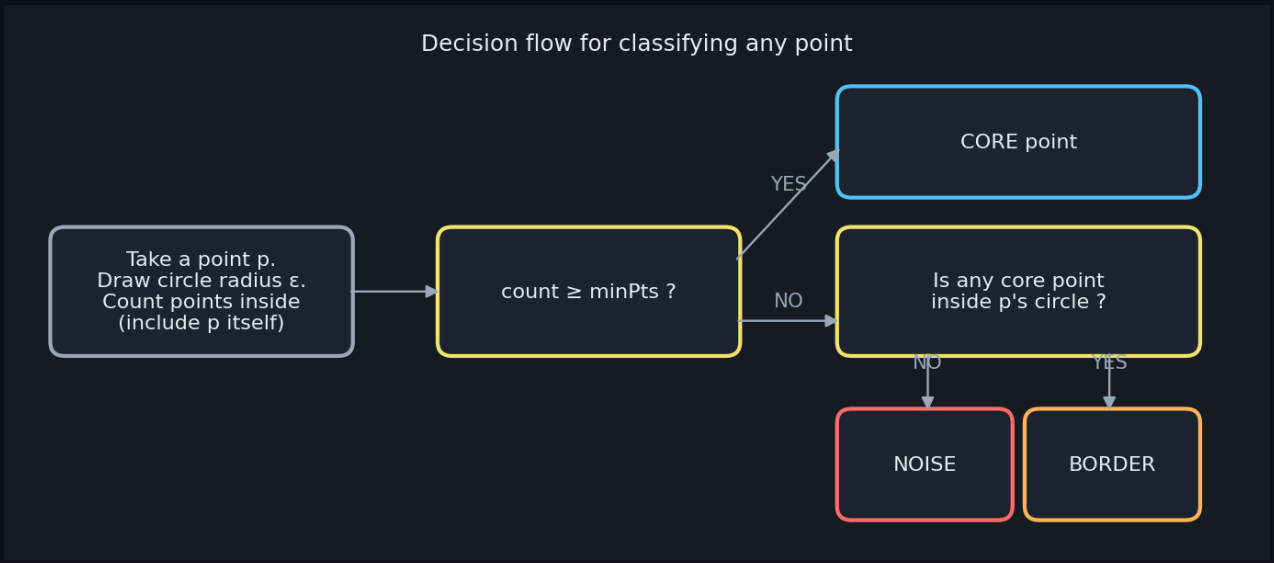


Figure A5 — The decision flow for classifying any point.

On a real dataset you see exactly this categorisation: the points that form the body of a cluster are core points, the points on the periphery of each cluster are border points, and the far-flung isolated points are noise.



Figure A6 — A real DBSCAN run on half-moons plus random outliers: filled dots = core, rings = border, red crosses = noise.

A5 · Density-connected points (how clusters grow)

One last concept before the algorithm. If two points A and B are **density-connected**, it simply means that you may put both of them in **one cluster**. The real question is how we know that two points are density-connected.

Rule: A and B are density-connected if they are linked through a chain of **core points** in which every pair of adjacent core points is at a distance ≤ ε. Think of stepping stones across a river: every stone must be solid (core) and the gap between neighbouring stones must be jumpable (≤ ε).

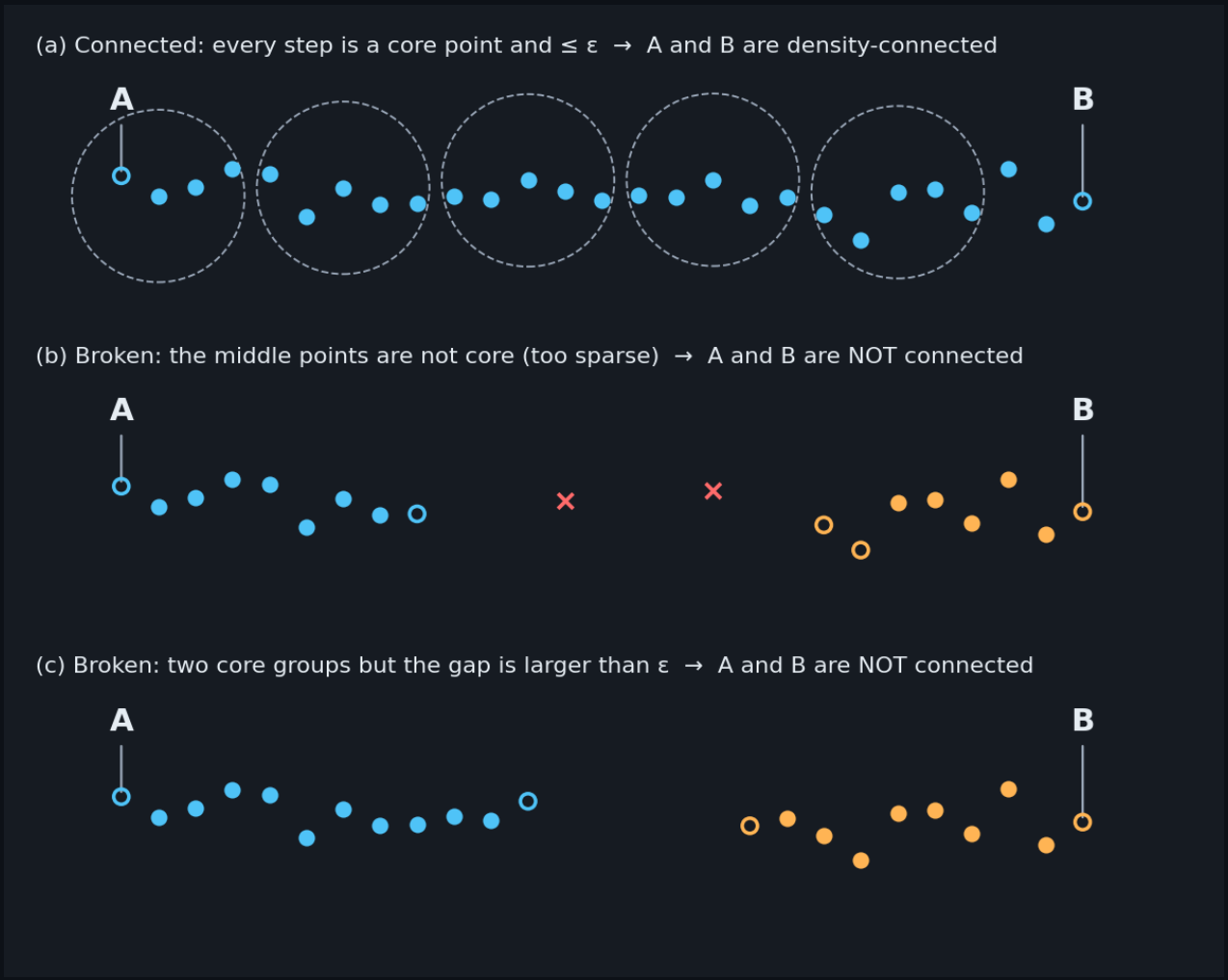


Figure A7 — (a) a continuous chain of core points links A and B. (b) Middle points are not core, the chain breaks. (c) All points are core but the gap exceeds ε, the chain breaks. Dashed circles are ε-neighbourhoods.

The path **breaks** in exactly two cases, and then A and B must go to different clusters:

1. **A point in the middle is not core** — it is a border or a noise point, so it cannot pass the connection on.
2. **Two adjacent core points are farther apart than ε** — even though both are core, the jump is too long.

The formal definitions (pro level)

Term	Definition
Directly density-reachable	q is directly density-reachable from p if **$q \in N_\varepsilon(p)$** and **p is a core point**. (Not symmetric: a border point is reachable from a core point, never the other way round.)
Density-reachable	q is density-reachable from p if there is a chain $p = p_1, p_2, \ldots, p_n = q$ where each p_{i+1} is directly density-reachable from p_i.
Density-connected	p and q are density-connected if there is a point o from which **both** are density-reachable. (This is symmetric.)
Cluster	A non-empty set of points that is *maximal* (anything density-reachable from a member is also a member) and *connected* (any two members are density-connected).

> **▲ A subtle point: border points can be ambiguous**
>
> A border point may lie within ε of core points from **two different clusters**. The result then depends on which cluster reaches it first, i.e. on the order of the data. Core points and noise points are always labelled the same way regardless of order; only such border points can change. This is a known, minor quirk of DBSCAN.

A6 · The DBSCAN algorithm, step by step

Now we have all the ingredients: minPts, ε, the three point types and density-connectivity. The whole algorithm executes in **four steps**:

1. **Label every point.** For each point count the points in its ε-circle and mark it core, border or noise (Figure A5).
2. **Pick an unvisited core point** and open a **new cluster** with it; add all points inside its ε-neighbourhood.
3. **Expand.** Every *core* point that was just added also adds *its* ε-neighbours (a chain reaction). Border points join the cluster but do not expand it. When nothing new can be added, the cluster is finished.
4. **Repeat** from step 2 for the next unvisited core point to build the next cluster. Points that never joined any cluster are **noise** (label −1).

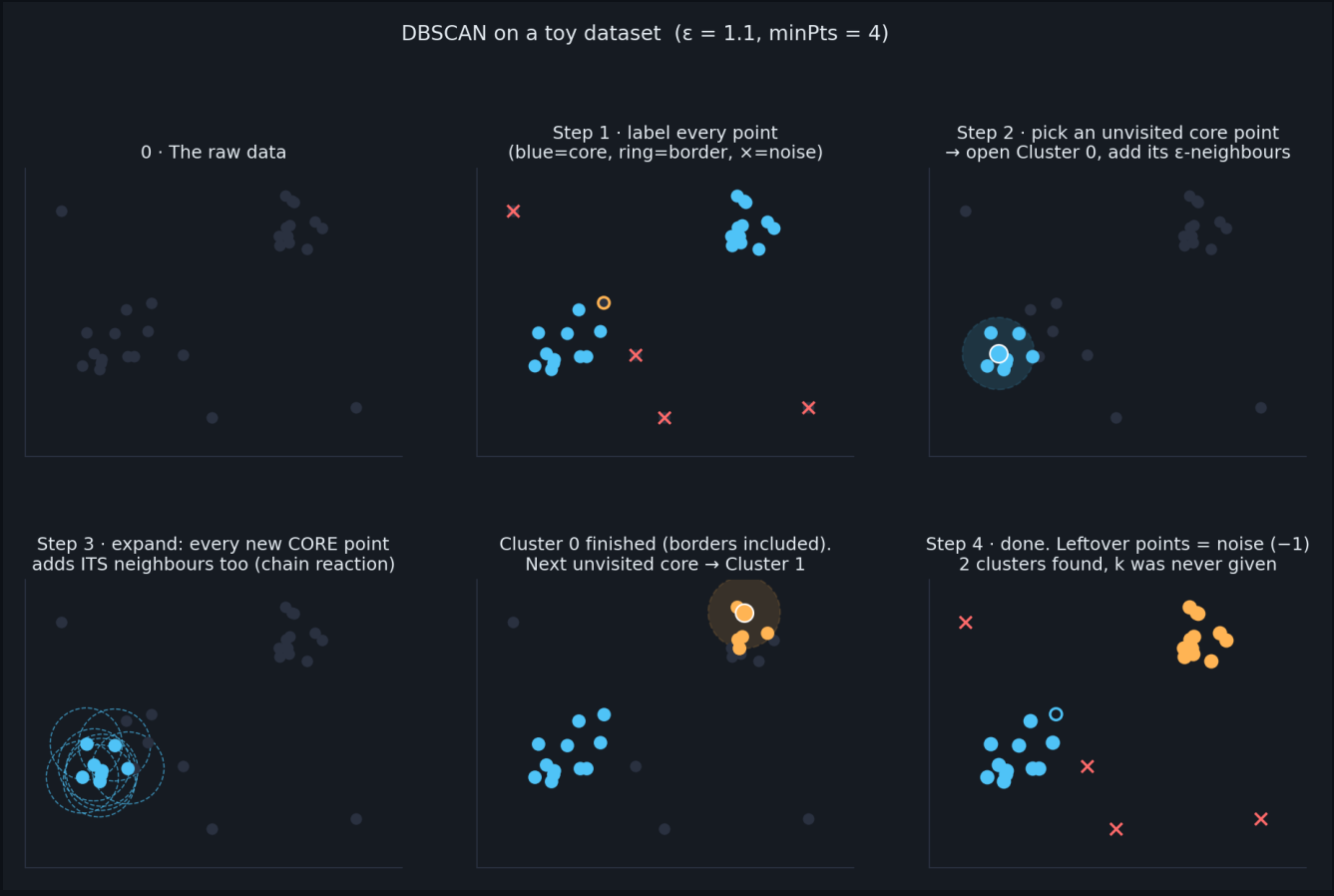


Figure A8 — The four steps on a toy dataset (ε = 1.1, minPts = 4). The number of clusters (2) is an output, not an input.

Notice what never happened: nobody told the algorithm that there were two clusters. They emerged because the sparse space between the two groups could not be crossed by a chain of core points.

Pseudocode

pseudocode

```
DBSCAN(D, eps, minPts):
    label every point as "unvisited"
    cluster_id = 0
    for each point p in D:
        if p is already in a cluster: continue
        N = neighbours of p within eps
        if |N| < minPts: continue           # not core (may become border later)
        cluster_id += 1                     # p is core -> start a new cluster
        put p in cluster cluster_id
        queue = N
        while queue is not empty:
            q = queue.pop()
            if q has no cluster: put q in cluster cluster_id
            if q is core: add q's unvisited neighbours to queue   # expansion
    every point without a cluster is NOISE (-1)
```

DBSCAN from scratch in Python (about 20 lines)

dbscan_scratch.py

```python
import numpy as np

def dbscan_scratch(X, eps, min_pts):
    n = len(X)
    # pairwise Euclidean distances (fine for small n: O(n^2) memory)
    D = np.linalg.norm(X[:, None, :] - X[None, :, :], axis=2)
    neighbours = [np.where(D[i] <= eps)[0] for i in range(n)]       # includes the point itself
    is_core = np.array([len(nb) >= min_pts for nb in neighbours])   # step 1
    labels = np.full(n, -1)                                         # -1 = noise
    cluster_id = 0
    for i in range(n):
        if labels[i] != -1 or not is_core[i]:
            continue                        # already clustered, or not core
        labels[i] = cluster_id              # step 2: new cluster from an unvisited core point
        queue = [i]
        while queue:                        # step 3: expansion
            p = queue.pop(0)
            if not is_core[p]:
                continue                    # border points join but do not expand
            for q in neighbours[p]:
                if labels[q] == -1:
                    labels[q] = cluster_id
                    queue.append(q)
        cluster_id += 1                     # step 4: go on with the next cluster
    return labels, is_core
```

Run this on the same data as scikit-learn's DBSCAN and the labels agree (up to renaming of clusters and the border-point ambiguity described in A5). Writing it yourself once is the best way to remember the algorithm.

Complexity

The expensive part is the ε-neighbourhood query, done once per point. With a brute-force search this is $O(n^2)$ time overall; with a spatial index (k-d tree or ball tree) each query costs about $O(\log n)$, so the total is about **$O(n \log n)$** in low dimensions. In high dimensions the indices stop helping and it drifts back toward $O(n^2)$. scikit-learn picks the algorithm via `algorithm='auto'`.

A7 · DBSCAN in scikit-learn (the examples from the lecture)

scikit-learn makes everything easy: create an object, give it the two hyperparameters, call `fit`, read `labels_`. The first example is taken straight from scikit-learn's documentation: six points, of which the first three are close together, the next two are close together, and the last one is far away.

example 1

```
import numpy as np
from sklearn.cluster import DBSCAN

X = np.array([[1, 2], [2, 2], [2, 3],
              [8, 7], [8, 8], [25, 80]])

db = DBSCAN(eps=3, min_samples=2)    # the two hyperparameters
db.fit(X)
print(db.labels_)                    # the entire clustering result lives here
```

```
[ 0  0  0  1  1 -1]
```

Six points: the first three form cluster 0, the next two form cluster 1, and the last point is noise, so DBSCAN gives it the label **−1**. A deliberately tiny example makes the behaviour easy to see.

Experiment 1 — raise min_samples from 2 to 3 (lecture's quick question)

Pause and predict. Only the first three points still have ≥ 3 points (themselves included) within ε = 3, so they remain a cluster. The two points (8,7), (8,8) can never have 3 points in their circle because only two points exist there: they are not core, and they are not border either (no core point is near) → **noise**.

```
min_samples=3  ->  [ 0  0  0 -1 -1 -1]
```

Experiment 2 — raise eps to 10

Increasing ε makes the circle bigger, i.e. the neighbourhood bigger. Now the circle of one point reaches the other group, so all five nearby points end up in **one cluster**; the far point stays noise.

```
eps=10, min_samples=2  ->  [ 0  0  0  0  0 -1]
```

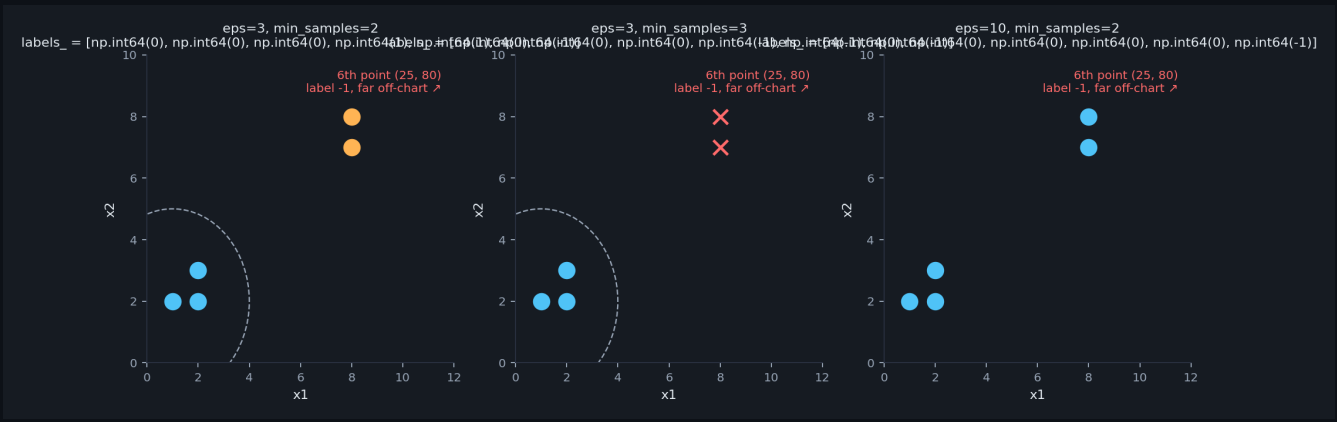


Figure A9 — The three runs on the same six points (the sixth point, (25, 80), is off-chart). ε and minPts are hyperparameters: vary them and watch the clustering change.

A proper example: concentric circles

example 2

```
from sklearn.datasets import make_circles
from sklearn.cluster import KMeans, DBSCAN

Xc, _ = make_circles(n_samples=500, factor=0.4, noise=0.05, random_state=0)
kmeans_labels = KMeans(n_clusters=2, n_init=10).fit_predict(Xc)    # cuts the rings in half
dbscan_labels = DBSCAN(eps=0.15, min_samples=5).fit_predict(Xc)    # recovers the two rings
```

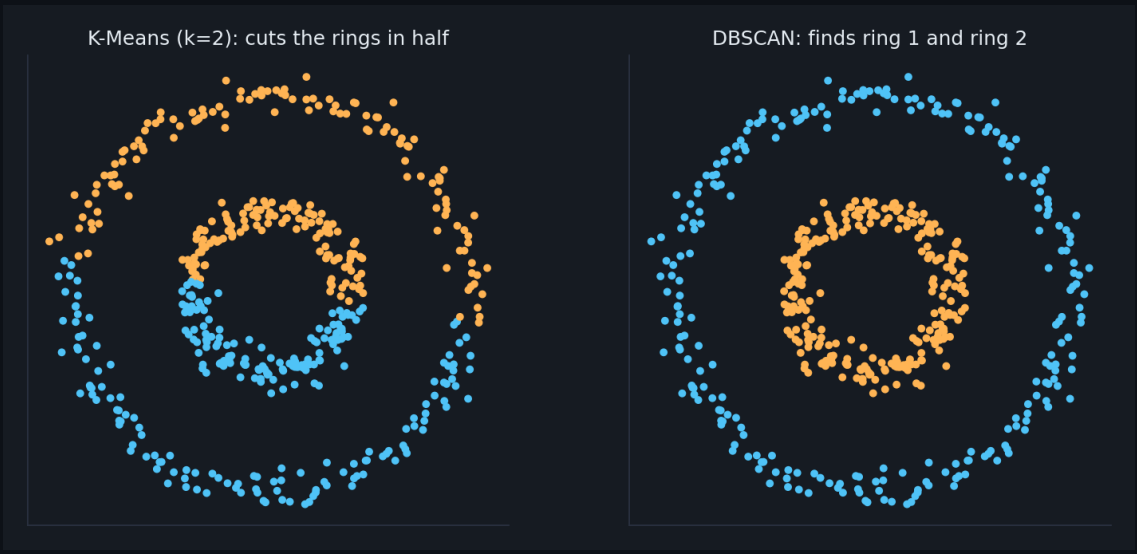


Figure A10 — K-Means cannot separate the rings; DBSCAN does not care about shape and size, it only cares about density.

Take the code, change the parameters, try other datasets (`make_moons`, `make_blobs`) — you will learn a lot.

Useful attributes and parameters (pro)

Item	What it gives you
`labels_`	Cluster index per point, **−1 = noise**.
`core_sample_indices_`	Indices of the core points. Border points = clustered points that are not in this list.
`components_`	The coordinates of the core points.
`fit_predict(X)`	Fit and return labels in one call. There is **no** `predict` for new data (see A9).
`metric`	Distance function: 'euclidean' (default), 'manhattan', 'cosine', 'precomputed' …
`algorithm`, `leaf_size`, `n_jobs`	How neighbours are searched ('auto', 'kd_tree', 'ball_tree', 'brute') and how many CPU cores to use.
`sample_weight` (in fit)	Give some points more weight when counting towards min_samples.

counting clusters, core, border and noise points

```
import numpy as np
db = DBSCAN(eps=0.15, min_samples=5).fit(Xc)
labels = db.labels_
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise    = int(np.sum(labels == -1))

core_mask = np.zeros(len(labels), dtype=bool)
core_mask[db.core_sample_indices_] = True
n_core   = core_mask.sum()
n_border = ((~core_mask) & (labels != -1)).sum()
print(n_clusters, n_core, n_border, n_noise)
```

A8 · Pro level: choosing ε and minPts, scaling and metrics

The lecture warns that DBSCAN is very sensitive to its hyperparameters. This section gives you a practical method instead of guessing.

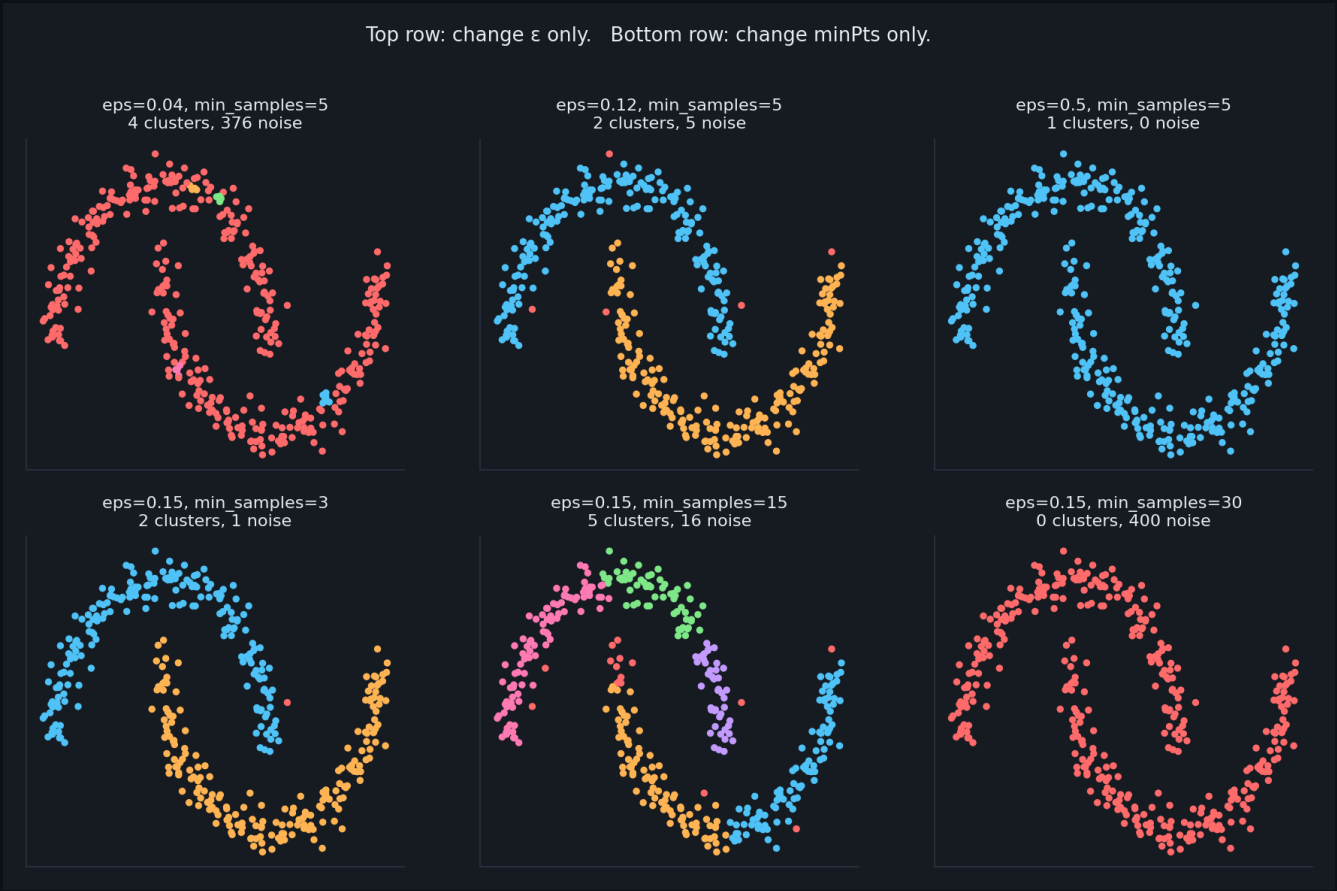


Figure A11 — Real effect of the hyperparameters on half-moons. Top: ε too small → almost everything is noise; ε right → 2 clusters; ε too large → everything merges. Bottom: minPts changes the result too — larger values fragment clusters and finally turn everything into noise.

Step 0 — scale your features

ε is one distance for *all* features. If one feature is in the thousands (salary) and another is 0–1, the big one dominates every distance. Always standardise (`StandardScaler` or `MinMaxScaler`) before DBSCAN.

Step 1 — choose minPts

- **Rule of thumb:** minPts ≥ D + 1 where D is the number of dimensions; the common choice is **minPts = 2 · D** (so 4 for 2-D data).
- For **noisy** data or large datasets use a larger value (e.g. 10–50) — larger minPts means more robust clusters and more noise points.
- minPts = 1 would make every point a core point (useless); minPts = 2 behaves like single-linkage clustering.

Step 2 — choose ε with the k-distance plot

For each point compute the distance to its *k*-th nearest neighbour with k = minPts, sort the values and plot them. Points inside clusters have small k-distances; noise points have large ones. The plot has a “knee”: the distance at the knee is a good ε.

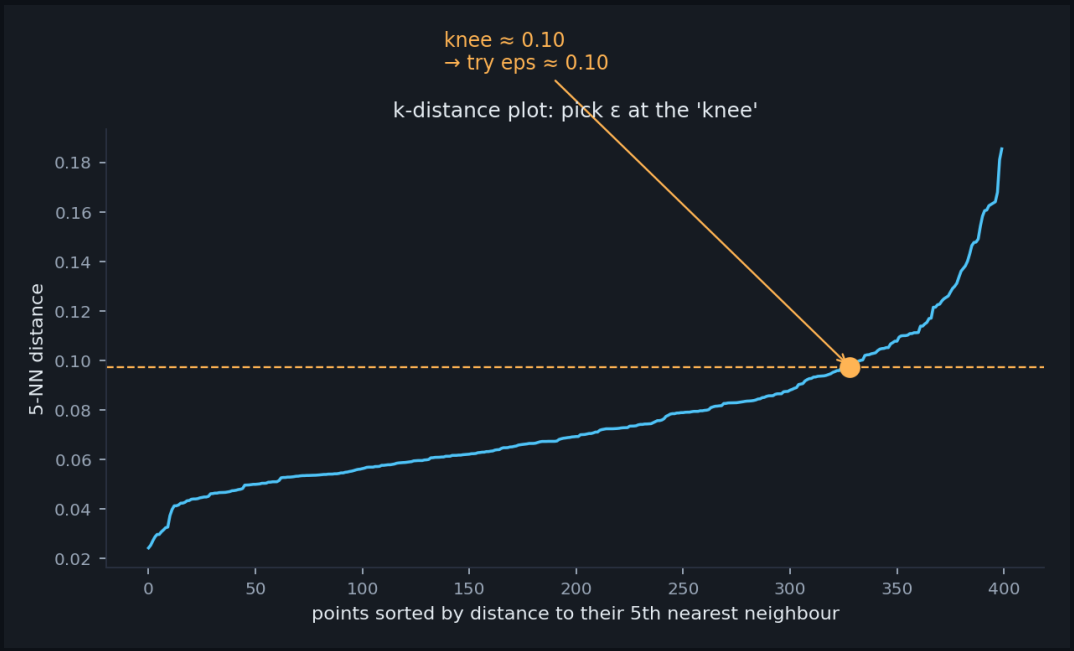


Figure A12 — k-distance plot for k = 5 on the moons data; the knee at ≈ 0.10 suggests an ε in the range 0.10–0.15 (the value that worked in Figure A11).

k-distance plot

```
from sklearn.neighbors import NearestNeighbors
import numpy as np, matplotlib.pyplot as plt

k = 5                                                  # = min_samples
dist, _ = NearestNeighbors(n_neighbors=k).fit(X).kneighbors(X)
kdist = np.sort(dist[:, -1])                           # distance to the k-th neighbour
plt.plot(kdist); plt.ylabel(f"{k}-NN distance"); plt.show()   # read eps at the knee
```

Step 3 — evaluate and iterate

Use a small grid over ε and minPts and judge each result by the number of clusters, the percentage of noise (a very high noise share means ε is too small or minPts too big) and, if you can, an internal metric such as the silhouette score computed *without* the noise points.

small hyperparameter search

```
from sklearn.metrics import silhouette_score
for eps in [0.08, 0.12, 0.15, 0.2, 0.3]:
    for m in [3, 5, 10]:
        lab = DBSCAN(eps=eps, min_samples=m).fit_predict(X)
        mask = lab != -1
        if len(set(lab[mask])) > 1:
            s = silhouette_score(X[mask], lab[mask])
            print(f"eps={eps:<5} min_samples={m:<3} clusters={len(set(lab))-1:<3} noise={(~mask).mean():.0%}  silhouette
```

Distance metric and high dimensions

Euclidean distance is the default; for text or embeddings `metric='cosine'` is common, for geographic coordinates use the haversine metric with a ball tree. In very high dimensions all distances become similar (the **curse of dimensionality**), so “density” loses meaning — reduce dimensions first (PCA, UMAP) and then cluster.

A9 · Advantages and limitations

Advantages

1. **Robust to outliers.** It has a built-in ability to identify outliers: noise points are detected separately, which is why DBSCAN is also used for **anomaly detection**. K-Means was not good at this.

2. **No need to specify the number of clusters.** The algorithm works it out on its own (in the lecture's examples two clusters appear without telling it). One big headache of K-Means disappears.
3. **Arbitrarily shaped clusters.** Give it moons, rings, snakes — shape does not matter because the idea is based on density.
4. **Only two hyperparameters** (ε and minPts), unlike algorithms where you must remember many.

Limitations

1 · Very sensitive to the hyperparameters. A small change in ε or minPts can give a very different clustering (Figure A11). You must choose them carefully — use A8.

2 · Clusters of different densities. One ε is applied to the whole dataset. If one cluster is tightly packed and another is loose, no single ε suits both.

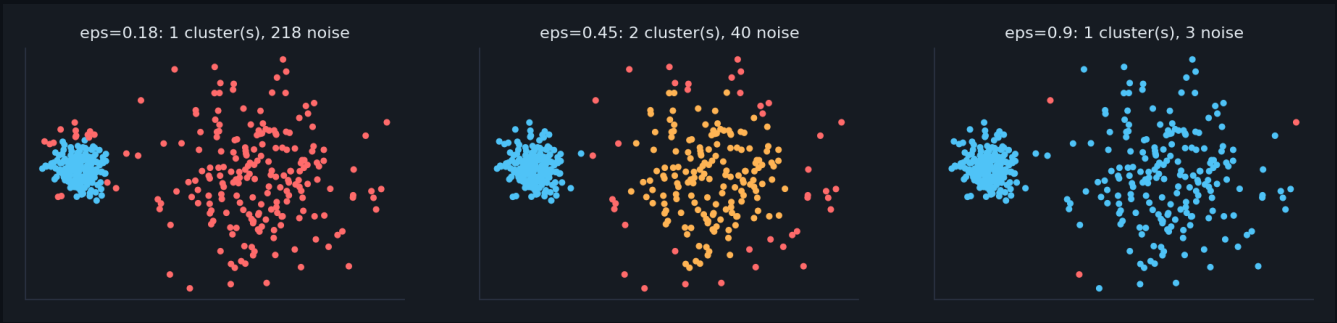


Figure A13 — Same data, three ε values. A small ε keeps the dense cluster but turns the sparse one into noise; a large ε captures the sparse one but merges both clusters.

3 · No predict function. DBSCAN only clusters the training data; there is no `predict`. If a new point arrives tomorrow it cannot say which cluster it belongs to — you would have to redo the calculations (re-cluster). It is useful for clustering, not for prediction.

> **✓ Workarounds and alternatives (extra)**
>
> **Predict a new point:** find the nearest *core* sample; if it is within ε use its cluster label, otherwise call the point noise (a few lines with `NearestNeighbors` on `db.components_`).
>
> **Different densities:** use **OPTICS** or **HDBSCAN** (`sklearn.cluster.HDBSCAN` in recent versions), which explore many ε values at once.
>
> **Huge data:** use a sample, a spatial index, or an approximate-neighbour library.

	K-Means	DBSCAN
Number of clusters	You must give k	Found automatically
Cluster shape	Round / convex	Any shape
Outliers	Pull centroids; every point is forced into a cluster	Labelled noise (−1)
Different densities	Handled moderately	Weak (single ε)
Predict new points	Yes (nearest centroid)	No built-in predict
Hyperparameters	k (+ initialisation)	ε, minPts (sensitive)
Speed	Very fast, O(n·k·iterations)	O(n log n) with an index, O(n²) worst case

A10 · Build solid intuition: the interactive visualization tool

The lecture ends with a “very kick-ass resource”: a website that shows DBSCAN running step by step. Search for **“Visualizing DBSCAN Clustering”** (the well-known interactive demo by Naftali Harris). It offers about 9–10 datasets (for example **Smiley Face**). You set ε and minPts with sliders and press **Go**; you then watch the circles move forward, add points to the cluster, stop, and start a new cluster — exactly Steps 2–4 of A6.

✓ How to practise with it (the lecturer’s own method)

Try **every** dataset the site offers and keep a notebook of observations: which ε / minPts gave the expected clusters, where it broke and why.

For each dataset deliberately set ε too small, then too large, then minPts too large, and write down what you see (compare with Figure A11).

Predict the outcome before pressing Go. Predicting first is what builds intuition.

Suggested practice plan. (1) Re-run every figure in this part with your own code. (2) Implement DBSCAN from scratch and compare labels with scikit-learn. (3) Cluster a real dataset (customer data, GPS points) and inspect the noise points. (4) Compare DBSCAN with K-Means and HDBSCAN on the same data.

A11 · Cheat-sheet and interview questions

● DBSCAN cheat-sheet

Input: data, ε, minPts. **Output:** labels (−1 = noise). **Core:** $|N_\varepsilon(p)| \geq$ minPts. **Border:** not core but has a core neighbour. **Noise:** neither.

Cluster = maximal set of density-connected points. Core points chain together when within ε; border points hang on the edge.

Tune: scale features → minPts ≈ 2·D → ε from the k-distance knee → check noise % and cluster count.

Strengths: no k, any shape, finds outliers. **Weaknesses:** sensitive hyperparameters, varying densities, no predict, high dimensions.

Typical interview questions (with short answers)

Question	Answer
Why DBSCAN instead of K-Means?	K-Means needs k, is outlier-sensitive and assumes spherical clusters; DBSCAN needs no k, flags outliers and finds arbitrary shapes.
Full form of DBSCAN?	Density-Based Spatial Clustering of Applications with Noise.
Name the two hyperparameters.	ε (neighbourhood radius) and minPts (minimum points to be dense).
Define core, border and noise points.	Core: ≥ minPts in the ε-neighbourhood. Border: fewer than minPts but within ε of a core point. Noise: neither.
What does density-connected mean?	Two points joined by a chain of core points whose consecutive distances are ≤ ε; they belong to one cluster.
What happens if ε is too small / too large?	Too small: most points become noise, many tiny clusters. Too large: clusters merge into one.
Why does DBSCAN have no predict()?	Clusters are defined by the training data’s density; a new point changes densities, so you must re-run (or approximate using the nearest core point).
When does DBSCAN fail?	Clusters of very different densities, very high dimensions, badly scaled features.
How do you pick ε?	k-distance plot with k = minPts; take the knee.
Time complexity?	$O(n^2)$ brute force, ≈ $O(n \log n)$ with a k-d / ball tree.

PART B — Optuna: smart hyperparameter tuning

What you will be able to do after this part. Explain why Grid Search and Random Search hit a wall and how Bayesian optimization (TPE) is smarter, use the five key Optuna terms (study, trial, trial parameters, objective function, sampler), tune a model in a handful of lines, switch between TPE / Random / Grid samplers, read Optuna's five visualizations, build dynamic search spaces with define-by-run (choosing the *algorithm itself* as a hyperparameter), and use professional features such as pruning, persistent storage and parallel workers.

> **▶ About the source and the figures**
> This part is built from the second lecture transcript ("…today we are going to talk about Optuna"). Numbers marked **"in the video"** are the lecturer's results on his dataset (the Water Potability CSV, 537 + 231 rows after the split). Because that file and the Optuna package are not available where this PDF was produced, the figures were generated by **re-running the same experiment on a synthetic 768 × 8 classification dataset** with the same split sizes, using a small hand-written TPE sampler that follows Optuna's default settings (10 random start-up trials, top-quantile "good" set, 24 candidates), plus Random and Grid search. The five "Optuna plots" are redrawn from that run in the same style. Exact numbers on your data will differ — the *shapes* and the *reasoning* are what matter.

B1 · Why Optuna? The problem with Grid Search and Random Search

First a quick refresher. **Parameters** are learned by the model from data (e.g. tree split thresholds). **Hyperparameters** are settings *you* choose before training (e.g. `max_depth`, `n_estimators`). Hyperparameter tuning means finding the values that give the best score. Optuna is a hyperparameter-tuning framework you install with `pip`; it lets you run large tuning jobs quickly and is one of the preferred ways of doing it in industry today. Until now you had two tools: `GridSearchCV` and `RandomizedSearchCV`. Why do we need a third?

Grid Search — try every combination

Grid Search is straightforward (the lecturer jokes it has a "Sunny Deol approach": brute force). Take a Random Forest with max_depth ∈ {1,…,5} and n_estimators ∈ {50,100,150,200,250}. That is 5 × 5 = 25 combinations. It trains a model for each one and measures accuracy on validation data:

Combination	max_depth	n_estimators	Accuracy (example)
1	1	50	71 %
2	1	100	69 %
3	1	150	74 %
…	…	…	…
25	5	250	…

When all 25 are done it picks the largest accuracy; the matching values are the "best parameters". Given enough time and compute, Grid Search *will* find the best point on your grid. The problem is the cost: with n_i candidate values for hyperparameter i, the number of model trainings is

$$N_{\text{grid}} = \prod_{i=1}^{h} n_i$$

Widen max_depth to 1–20 and it becomes 20 × 5 = **100** trainings instead of 25. Add a third hyperparameter (for example `min_samples_split`) and the combinations multiply again. And this is a tiny dataset: with deep learning, where one training run already takes hours, 100 runs are simply impossible. Trying all combinations is often not feasible.

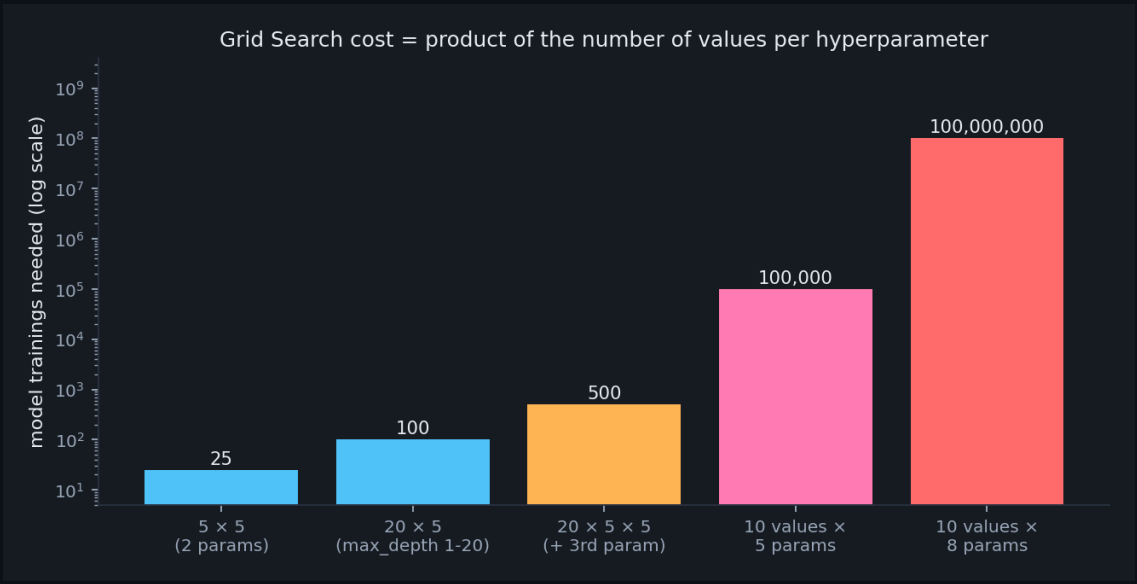


Figure B1 — The number of trainings Grid Search needs grows multiplicatively (log scale). Eight hyperparameters with ten values each would need a hundred million trainings.

Random Search — try a few random combinations

Random Search simplifies: “I will not try all combinations, I will randomly try some.” You say “try 5 combinations” and it samples 5 random points from the grid (or from continuous ranges), trains 5 models and reports the best of those. The **advantage** is saved computation. The **disadvantage** is obvious: the best value may have been hiding in a spot that Random Search never tried, and you live with a sub-par solution.

∑ How lucky is random search? (extra math)

Suppose the best 5 % of the search space gives near-optimal scores. The probability that n independent random trials hit it at least once is

$$P = 1 - (1 - 0.05)^n \quad \Rightarrow \quad n = 14:\ 51\%,\quad n = 30:\ 79\%,\quad n = 60:\ 95\%$$

So random search is surprisingly good, but it needs dozens of trials and — crucially — **each trial ignores everything learned from the previous ones**.

● The gap Optuna fills

Grid and Random Search are both **memoryless**: accuracy from one trial gives no help for choosing the next. Optuna uses a different kind of search, **Bayesian search (Bayesian optimization)**, in which what happened in earlier trials guides later trials. You try fewer combinations, but you try them *intelligently*.

B2 · Bayesian optimization from scratch

The lecturer does not want to go deep into the mathematics (it is a topic in itself, and he links a paper), but gives the rough idea. Let us rebuild it step by step.

Step 1 — there is a hidden function

If changing max_depth and n_estimators changes the accuracy, then there **must be a mathematical relationship** between them. For example if you change max_depth in a Random Forest from 5 to 25 the accuracy certainly changes (we do not know whether it goes up or down, but something changes); the same for n_estimators from 10 to 100. So accuracy is a *function* of the hyperparameters:

$$\text{accuracy} = f(\text{max_depth},\ \text{n_estimators})$$

We do **not know** f. The only thing we can do is *evaluate* it: give a max_depth and an n_estimators, train the whole model, and read the accuracy. Each evaluation is expensive. Our goal is the maximum of f:

$$x^* = \arg\max_{x \in \mathcal{X}} f(x), \qquad x = (\text{max_depth},\ \text{n_estimators})$$

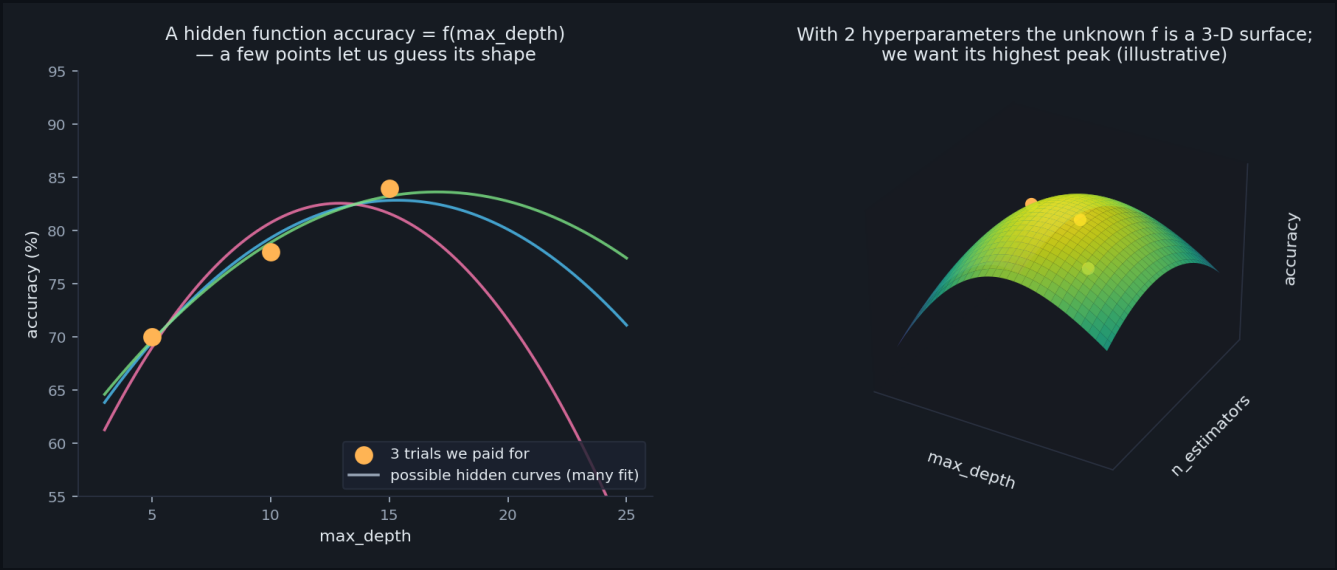


Figure B2 — Left: three evaluated points (the lecturer's 2-D example: max_depth 5 → 70, 10 → 78, 15 → 84) fit many possible hidden curves. Right: with two hyperparameters f is a 3-D surface (x = max_depth, y = n_estimators, z = accuracy) and we look for its peak.

The lecturer's 3-D example: you train with max_depth = 5, n_estimators = 100 and get accuracy 75 — that is one point on the surface. Then max_depth = 10, n_estimators = 150 gives 69 — another point. The surface need not be a parabola; it can be anything.

Step 2 — learn the surface bit by bit (the surrogate model)

Simplify to 2-D (only max_depth). Suppose we tried 5 → accuracy 70, then 10 → 78, then 15 → 84. With these three points we can already guess the rough shape of the curve and see that accuracy is still rising at 15. Bayesian search builds exactly such an **estimate of the hidden function** — called a **surrogate model** — from the points it has seen, and refines it after every new trial.

Step 3 — use the estimate to decide where to look next

Based on the 4 points it thinks "try this next"; with 5 points the graph is clearer so the next choice is better, and so on. Eventually the graph is understood well enough to find its maximum, and that is the end point. The decision rule is called the **acquisition function**.

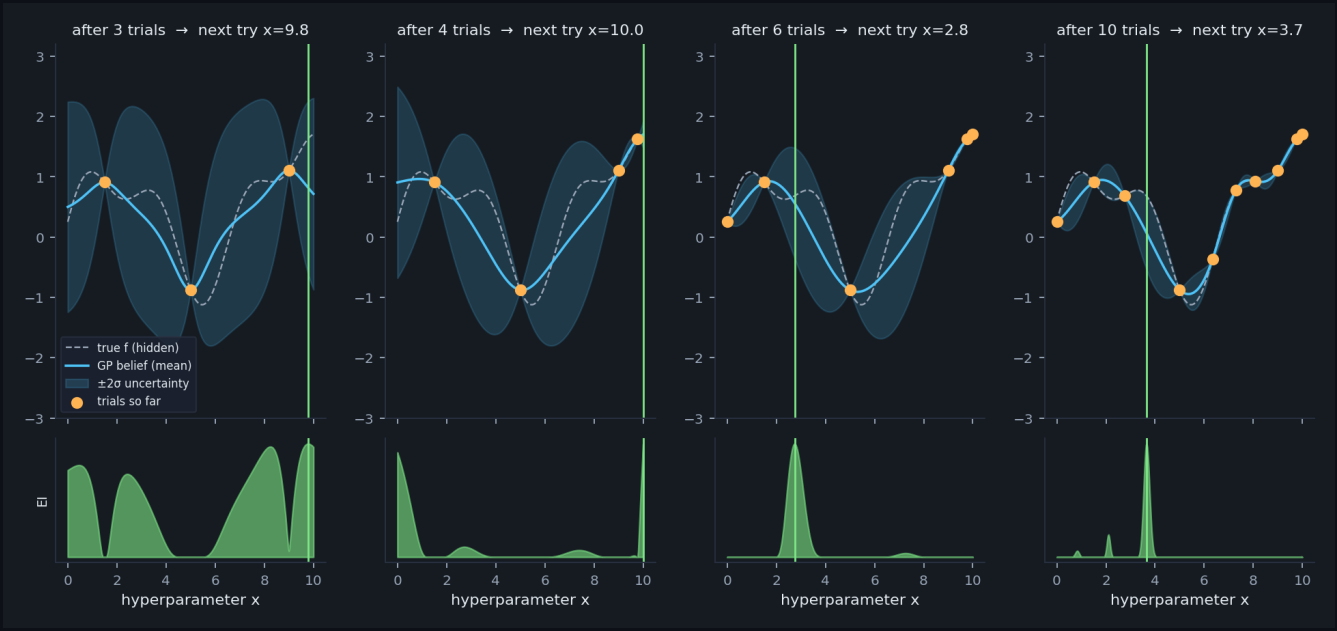


Figure B3 — Bayesian optimization on a 1-D toy function with a Gaussian-process surrogate. Top: the hidden function (dashed), the belief (blue line) and its uncertainty (band), the evaluated trials (orange). Bottom: Expected Improvement (EI); the green line marks its maximum = the next trial. Early on the next trial explores uncertain places; later it focuses on promising ones.

∑ The math behind the picture (optional, but this is "pro level")

Surrogate (Gaussian process). After n trials the model gives, for every untried x, a predicted mean μ(x) and an uncertainty σ(x):

$$f(x) \mid \text{data} \;\sim\; \mathcal{N}(\mu(x),\ \sigma^2(x))$$

Acquisition (Expected Improvement). Let f^+ be the best accuracy so far and $z = (\mu - f^+)/\sigma$. Then

$$\mathrm{EI}(x) = (\mu(x) - f^+)\,\Phi(z) + \sigma(x)\,\phi(z), \qquad z = \frac{\mu(x) - f^+}{\sigma(x)}$$

with Φ and φ the normal CDF and PDF. The next trial is x_{next} = argmax EI(x). The first term rewards points with a high predicted mean (**exploitation**), the second rewards points with high uncertainty (**exploration**). This balance is why Bayesian search neither gets stuck nor wanders aimlessly.

Grid vs Random vs Bayesian — side by side

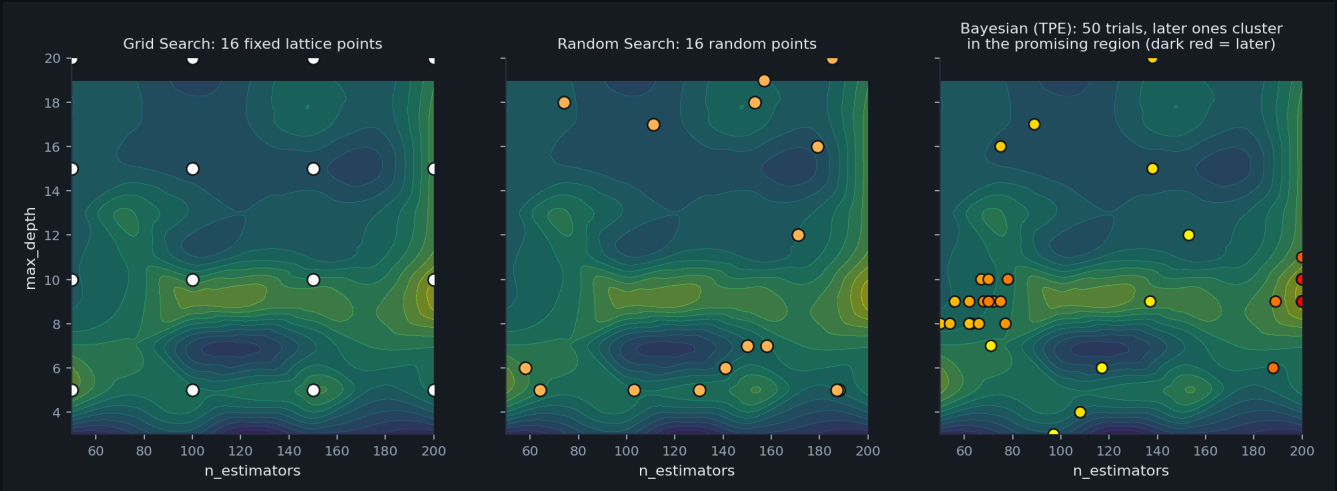


Figure B4 — The three strategies on the real accuracy surface of our reproduction (brighter = higher accuracy). Grid uses a rigid lattice, Random scatters, TPE (right) starts randomly and then concentrates its later trials (dark red) where accuracy is high.

	Grid Search	Random Search	Bayesian (Optuna TPE)
How it chooses	Every combination	Random combinations	Based on all previous results
Uses past results?	No	No	**Yes**
Cost	Explodes with # of hyperparameters	You choose n_trials	You choose n_trials
Risk	Too slow or too coarse	May miss the best region	Can get stuck in a local optimum (mitigated by exploration)

So the biggest power of Optuna: behind the scenes it uses **Bayesian optimization**, an intelligent way of searching for the best hyperparameter values — something you will not find in Grid Search or Random Search. For the original mathematics the lecturer recommends reading the paper he links (see References).

B3 · TPE — the sampler Optuna uses by default

The classic surrogate is a Gaussian process, which becomes slow with many trials and awkward with categorical or conditional hyperparameters. Optuna's default sampler is **TPE — Tree-structured Parzen Estimator** — which flips the question around. Instead of modelling p(score | x) directly it models p(x | score) using Bayes' rule.

- Sort all past trials by score and split them at a quantile γ into a **good** group (top γ share) and a **bad** group (the rest).
- Fit a smooth density to the hyperparameter values of each group with a **Parzen estimator** (a kernel-density estimate): l(x) for good, g(x) for bad.
- Draw many candidate values from l(x), and pick the candidate with the largest ratio l(x)/g(x): "very typical of good trials, very untypical of bad trials".

$$l(x) = \frac{1}{n_g} \sum_{i \in \text{good}} \frac{1}{h} K\left(\frac{x - x_i}{h}\right), \qquad x_{\text{next}} = \arg\max_{x \sim l} \frac{l(x)}{g(x)}$$

The ratio rule is not arbitrary: Bergstra et al. showed that Expected Improvement under this model is monotone in l(x)/g(x), so maximising the ratio maximises EI. "Tree-structured" means the search space may be a tree — some hyperparameters exist only if another one has a certain value (that is exactly Optuna's define-by-run feature in B8).

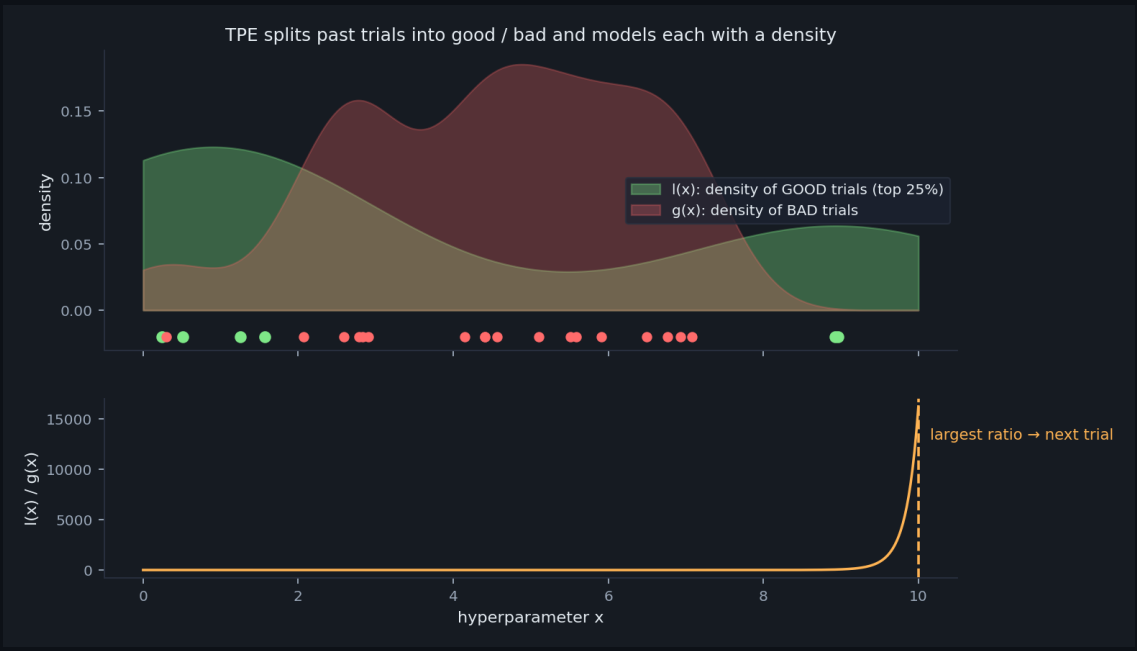


Figure B5 — TPE on a 1-D example: green = density of the good trials, red = density of the bad trials. The ratio l/g (bottom) peaks where good trials are common and bad trials rare; the next trial goes there.

✔ Optuna's TPE defaults (good to know)

The first **10 trials are random** (`n_startup_trials=10`) because there is no history yet; each later step evaluates **24 candidates** (`n_ei_candidates=24`); the "good" group is roughly the best 10 % of trials (capped at 25). Pass `seed=...` for reproducibility.

B4 · The five key terms (learn these before reading Optuna code)

Optuna code looks scary if you do not know five words. Learn them now.

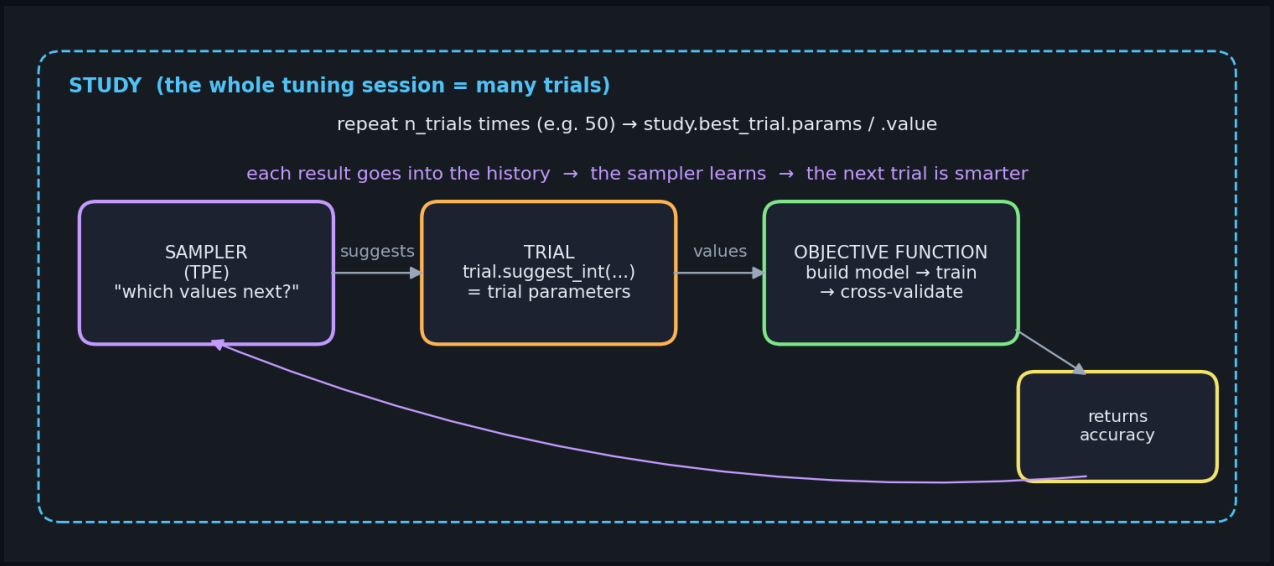


Figure B6 — How the five terms fit together. The sampler suggests values, a trial holds them, the objective function trains and scores a model, and the score flows back to the sampler.

1 · Study

The lecturer's setting: a dataset, an algorithm (Random Forest) and two hyperparameters (max_depth, n_estimators); we want the values that maximise accuracy. This **whole optimisation task is a study**. In Optuna's words: "an optimization session that encompasses multiple trials" — a collection of trials aimed at

optimising the objective function.

2 · Trial

To find the best hyperparameters you try different combinations: first max_depth 1 with n_estimators 50, then max_depth 2 with n_estimators 100, and so on. Every time you take one combination and train a model with it, that single run or experiment is a **trial**. Many trials make a study; after running them all you automatically have a *best trial* and, inside it, your best hyperparameter values.

3 · Trial parameters

The hyperparameter values used during one particular trial — e.g. max_depth = 2 and n_estimators = 100 in that trial — are that trial's **trial parameters**.

4 · Objective function

The relationship we search for. We cannot see it, but we can do this much: give a max_depth and an n_estimators, train the model, and get an accuracy back. That **whole process — values in, accuracy out — is the objective function**. In code it is a Python function that receives a `trial` object and returns one number.

5 · Sampler

The most important component. It is the algorithm that suggests which hyperparameters should be evaluated next, based on the trials run so far. There are a lot of possible combinations; the sampler uses past data to tell you which to try next. The default is **TPE**, and it executes the Bayesian optimization behind the scenes.

Term	One-line meaning	Code you will see
Study	The whole tuning job	`study = optuna.create_study(...)`
Trial	One experiment with one set of values	`def objective(trial): ...`
Trial parameters	The values used in that trial	`trial.params`, `study.best_params`
Objective function	Values in → score out	`return score`
Sampler	Decides the next values (TPE by default)	`sampler=optuna.samplers.TPESampler()`

> **● Analogy: perfecting a cake recipe**
>
> **Study** = the whole project "find the best cake". **Trial** = baking one cake. **Trial parameters** = the sugar and baking time you used for that cake. **Objective function** = "bake it, ask the tasters, return their score". **Sampler** = the clever friend who looks at all previous cakes and says "the best ones had less sugar and 35 minutes — try 33 next".

B5 · Hands-on: your first Optuna study

Open a Google Colab notebook (you get free compute so tuning is quick). The lecturer's advice: code along line by line and keep the notebook next to the video.

Step 1 — install and load

setup

```
!pip install optuna          # not pre-installed on Google Colab

import numpy as np, pandas as pd, optuna
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("water_potability.csv")      # classification dataset used in the video
```

Step 2 — fix the missing values (zeros mean “missing” in this dataset)

The dataset has a problem: missing values are represented by **0**. So first replace zeros with `np.nan`, then impute the NaNs — the lecturer simply uses the **mean** (no deep thought, a quick start).

cleaning

```
target = df.columns[-1]                       # assume the label is the last column
feature_cols = [c for c in df.columns if c != target]
df[feature_cols] = df[feature_cols].replace(0, np.nan)          # zeros -> NaN (features only!)
df[feature_cols] = df[feature_cols].fillna(df[feature_cols].mean())   # NaN -> column mean
print(df.isna().sum().sum())                  # 0 -> no missing values left
```

Step 3 — split and scale

split

```
X = df[feature_cols]; y = df[target]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)       # scaling is not really needed for Random Forest,
X_test  = scaler.transform(X_test)            # but it does no harm
print(X_train.shape, X_test.shape)            # in the video: (537, 8) and (231, 8)
```

> **▲ Pro tip: avoid data leakage**
> Imputing with the mean of the *whole* dataset before splitting leaks test-set information into training. The cleaner way is to split first and put the imputer and scaler into a `Pipeline` so that cross-validation re-fits them inside every fold (see B9).

Step 4 — the objective function (two jobs)

Job 1: define the search space — which hyperparameters, in which ranges. Here: n_estimators between 50 and 200, max_depth between 3 and 20. **Job 2: the training workflow** — build the model with the suggested values, train it with cross-validation (several trainings → several accuracies), and return the *mean* accuracy.

objective

```
def objective(trial):
    # --- job 1: search space ---------------------------------------------
    n_estimators = trial.suggest_int("n_estimators", 50, 200)    # every integer 50..200 is allowed
    max_depth    = trial.suggest_int("max_depth", 3, 20)
    # --- job 2: training workflow ----------------------------------------
    model = RandomForestClassifier(n_estimators=n_estimators,
                                   max_depth=max_depth,
                                   random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=3, scoring="accuracy")
    return scores.mean()                                          # the number Optuna will maximise
```

> **● What is that mysterious trial.suggest_int?**
> The `trial` object that Optuna hands to your function contains "the intelligence": when you call `trial.suggest_int(name, low, high)`, the **sampler** looks at the history and returns the value it believes is most promising (during the first trials it is random). Note that Optuna tries *every* integer — 51, 52, 65, 66 … — not just steps of 50 like a grid.

Step 5 — create the study and optimize (just two lines)

study

```
study = optuna.create_study(direction="maximize",                # we maximise accuracy
                            sampler=optuna.samplers.TPESampler()) # default anyway; written to be explicit
study.optimize(objective, n_trials=50)                           # run 50 trials
```

- **direction:** "maximize" for accuracy, R^2 score, F1 …; "minimize" for losses such as cross-entropy or mean squared error. It depends on the metric your objective returns.
- **sampler:** decides the next values; TPESampler triggers Bayesian optimization behind the scenes.
- **optimize(objective, n_trials=50):** tells Optuna what the objective function is and how many trials to run.

The whole flow: you define an objective function (hyper-values in, accuracy out). You create a study. The study creates 50 trials; in each trial it calls your objective with a `trial` object; the sampler (using past data) chooses the next values; a new Random Forest is built and trained; the accuracy goes back; the sampler decides again — iteratively 50 times. Just two lines of code plus one function.

Step 6 — read the output

```
[I ...] Trial 0 finished with value: 0.77... and parameters: {'n_estimators': 69,  'max_depth': 8}.
[I ...] Trial 1 finished with value: ...     and parameters: {'n_estimators': 122, 'max_depth': 20}.
[I ...] Trial 2 finished with value: ...     and parameters: {'n_estimators': 144, 'max_depth': 15}.
[I ...] Trial 3 finished with value: ...     and parameters: {'n_estimators': 197, 'max_depth': 9}.
...                                          (Trial 4 ... Trial 49)
```

In the video, Trial 0 picked n_estimators = 69 and max_depth = 8 and scored about 77 %. The sampler then proposed (122, 20), then (144, 15), then (197, 9): it is spreading points over the search space and collecting accuracies. As the points accumulate, the promising regions become visible and later trials concentrate there.

best results

```
print(study.best_value)       # best accuracy found      (video: ~0.78 on training data / CV)
print(study.best_params)      # {'n_estimators': 115, 'max_depth': 18}   (video)
print(study.best_trial.number)
```

Step 7 — train the final model and test it

final model

```
best = study.best_params
final_model = RandomForestClassifier(**best, random_state=42).fit(X_train, y_train)
print(final_model.score(X_test, y_test))      # video: ~0.75 on the test data -> a good starting point
```

Result	In the video	Our reproduction (synthetic data)
Best CV accuracy after 50 TPE trials	≈ 78 % (training data)	79.9 % (hand-written TPE, seed 0)
Best hyper-parameters	n_estimators = 115, max_depth = 18	n_estimators = 189, max_depth = 9
Accuracy on test data	≈ 75 %	81.4 %

Different datasets give different optima — 115/18 vs 189/9 — which is exactly why you tune per dataset. With more care in preprocessing and imputation the lecturer expects better results; the goal here is a good starting point.

B6 · Samplers: Bayesian, Random and Grid with the same objective

A great feature: Optuna has **many samplers**. The same objective function works with all of them; you only change the `sampler` argument. You do not need scikit-learn's search classes for Random or Grid search any more.

Random search

RandomSampler

```
study_rand = optuna.create_study(direction="maximize", sampler=optuna.samplers.RandomSampler())
study_rand.optimize(objective, n_trials=50)
print(study_rand.best_params, study_rand.best_value)
```

In the video Random Search also ended with n_estimators = 115 and max_depth = 18 and a test accuracy of 75 % — close to Bayesian search, because it is a small dataset and not much work went into it. On bigger problems you generally see a difference.

Grid search

GridSampler

```
search_space = {"n_estimators": [50, 100, 150, 200],      # defined OUTSIDE the objective
                "max_depth":    [5, 10, 15, 20]}
study_grid = optuna.create_study(direction="maximize",
                                 sampler=optuna.samplers.GridSampler(search_space))
study_grid.optimize(objective)                             # 4 x 4 = 16 trials, then it stops
```

Here no Bayesian search is applied — every combination is tried: 16 trials (0 to 15). In the video the grid result was a little worse than before: accuracy ≈ 74 %.

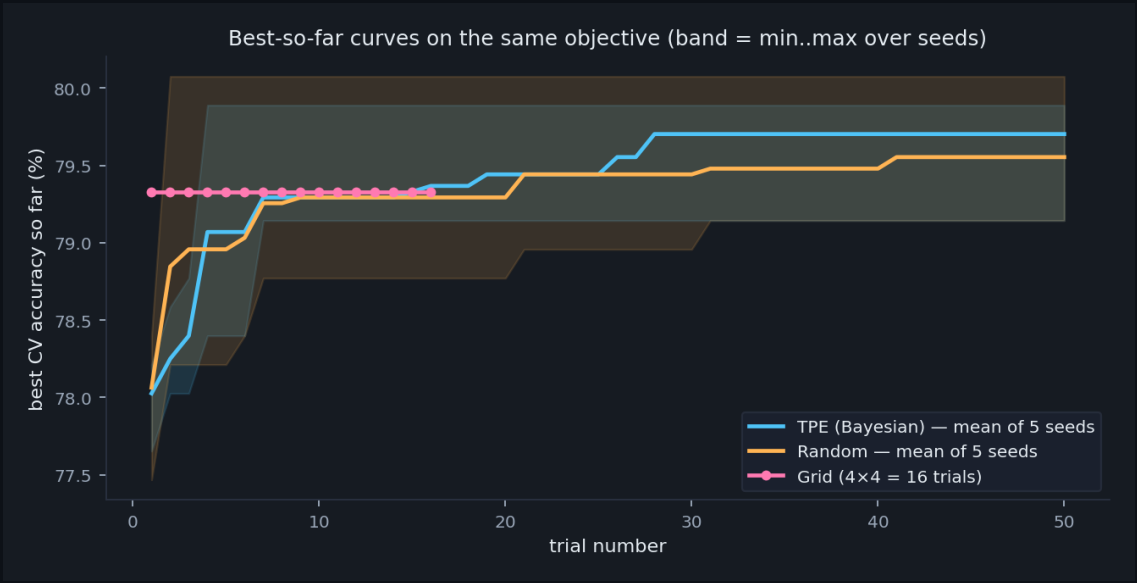


Figure B7 — Best CV accuracy found so far, versus the number of trials (our reproduction; band = best/worst of 5 seeds). TPE ends highest (79.7 % vs 79.6 % Random, 79.3 % Grid) — on this small, noisy 2-parameter problem the gaps are small; they widen with more hyperparameters and expensive models.

> **▲ Be honest about small differences**
> On a two-hyperparameter toy problem with accuracy measured on only a few hundred rows, differences of a few tenths of a percent are within noise. TPE's real advantage is **efficiency in large, expensive, many-dimensional search spaces**. Run several seeds before claiming a winner.

Other samplers (extra)

Sampler	Idea / when to use
`TPESampler` (default)	Bayesian, handles categorical and conditional spaces well; a great default.
`RandomSampler`	Baseline; also good for very cheap objectives.
`GridSampler`	Exhaustive over a small explicit grid.
`CmaEsSampler`	Evolution strategy; strong for continuous numeric spaces (needs the `cmaes` package).
`GPSampler`	Gaussian-process Bayesian optimization (as in Figure B3); good for few, expensive trials.
`NSGAIISampler`, `NSGAIIISampler`	Genetic algorithms for multi-objective studies (B9).
`QMCSampler`	Quasi-random low-discrepancy sequences, more even coverage than pure random.

B7 · Optuna's visualizations — see what happened

Optuna can plot the story of an optimisation, so you do not just get an answer, you can *understand* the search. After tuning (the video re-ran the TPE study first) import the plot functions and show them. Each returns an interactive Plotly figure.

`visualizations (needs plotly)`

```
from optuna.visualization import (plot_optimization_history, plot_parallel_coordinate,
                                  plot_slice, plot_contour, plot_param_importances)

plot_optimization_history(study).show()
plot_parallel_coordinate(study).show()
plot_slice(study).show()
plot_contour(study).show()
plot_param_importances(study).show()
```

1 · Optimization history plot

Trial number against objective value. In the video, accuracy rose steadily as trials increased but stopped improving after about the 31st trial. That tells you how many trials you really need to plan next time.

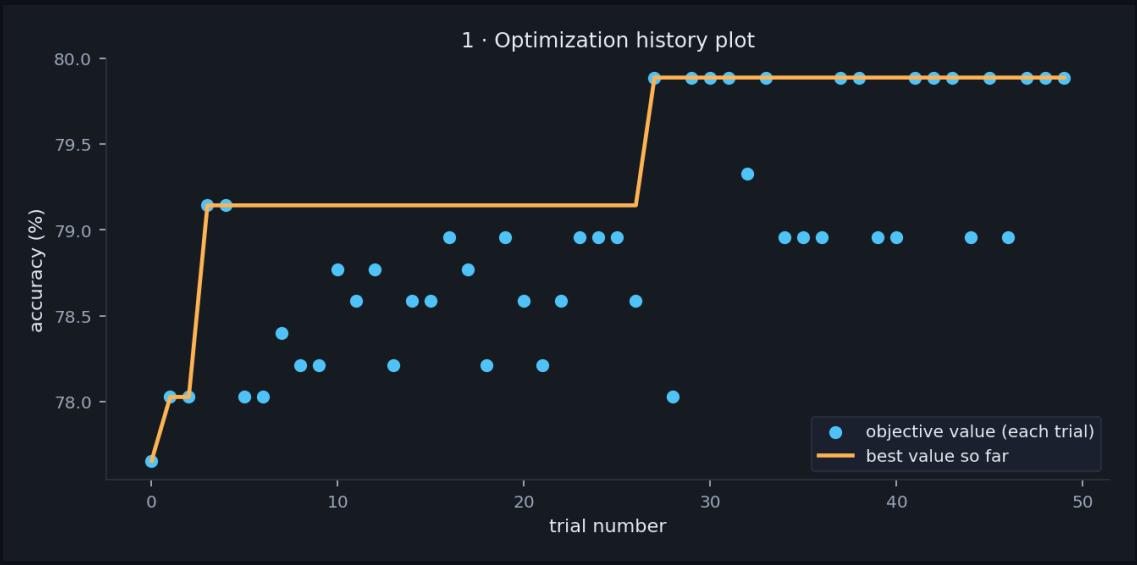


Figure B8 — Optimization history (our reproduction). Dots = each trial, line = best so far. Where the line goes flat, more trials are wasted effort.

2 · Parallel coordinate plot

Famous plot (also used a lot in MLflow). Three vertical axes: objective value (accuracy), max_depth and n_estimators. Each line is one trial and its colour shows the objective value. Where many lines pass through a region, many trials were run there, which means the sampler thought it was promising. In the video most lines lay between max_depth 12 and 20 and around n_estimators 100–150.

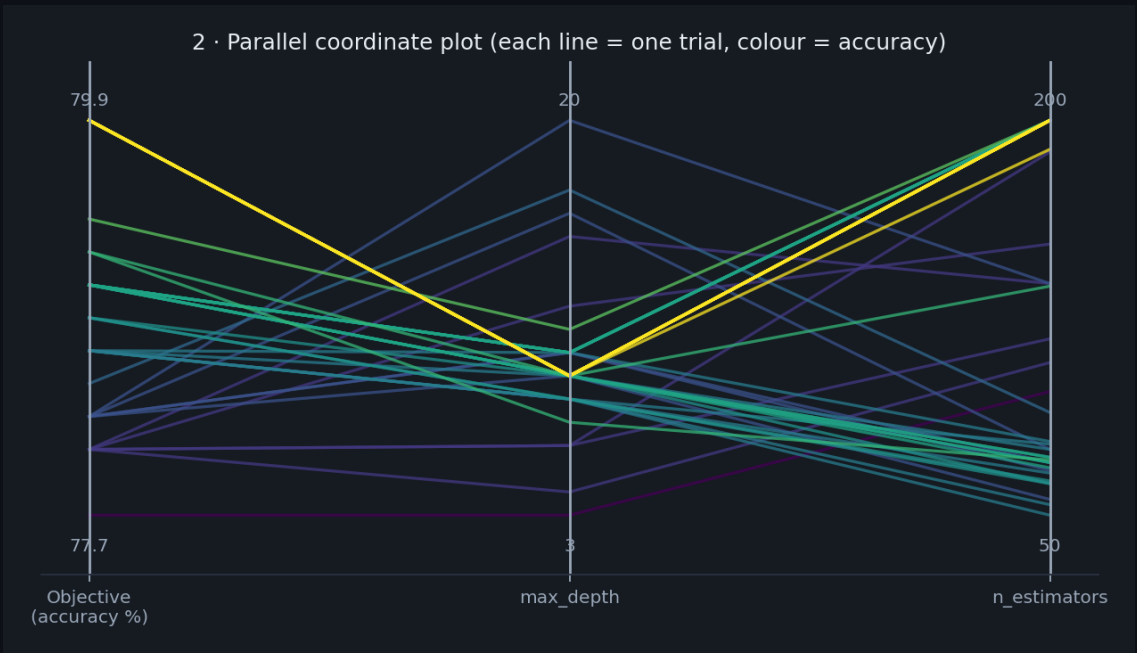


Figure B9 — Parallel coordinate plot (our reproduction). Bright lines = high accuracy; follow them to the axis values that produced them.

3 · Slice plot

Does the same thing one hyperparameter at a time: max_depth versus objective value, then n_estimators versus objective value. In the video many trials were between 15 and 20 for max_depth and between 100 and 150 for n_estimators.

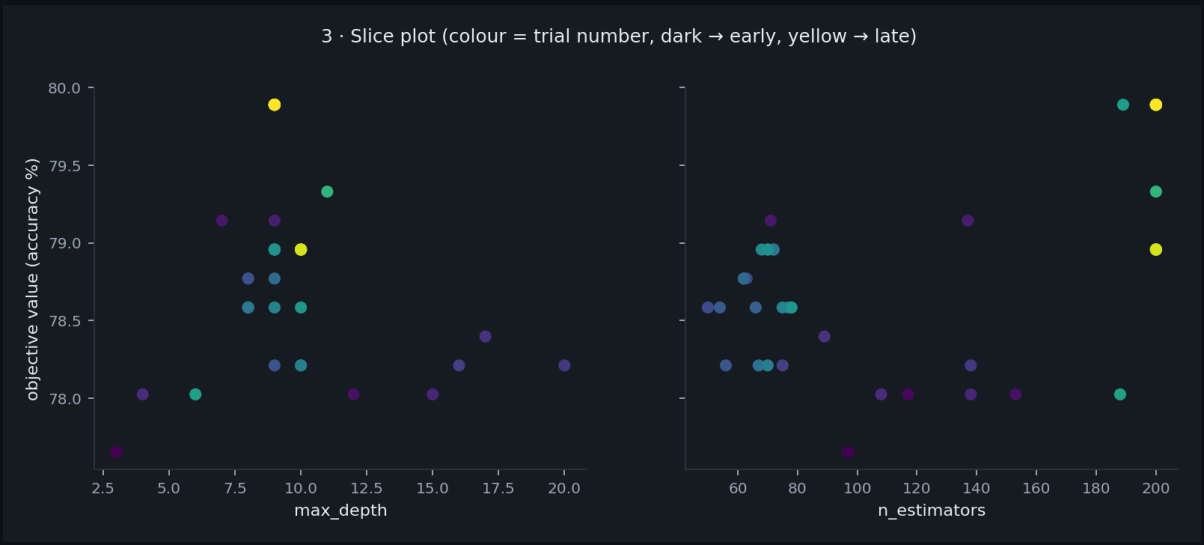


Figure B10 — Slice plot (our reproduction).

4 · Contour plot

x-axis = max_depth, y-axis = n_estimators, and the shaded surface = accuracy; the darker the blue, the higher the accuracy. It shows the **hotspots** of your search space: where the chance of finding the best configuration is highest. This is the picture from earlier: at first the sampler tries scattered points, learns that one region gives higher accuracy, and then stops spending trials elsewhere and puts all its energy there.

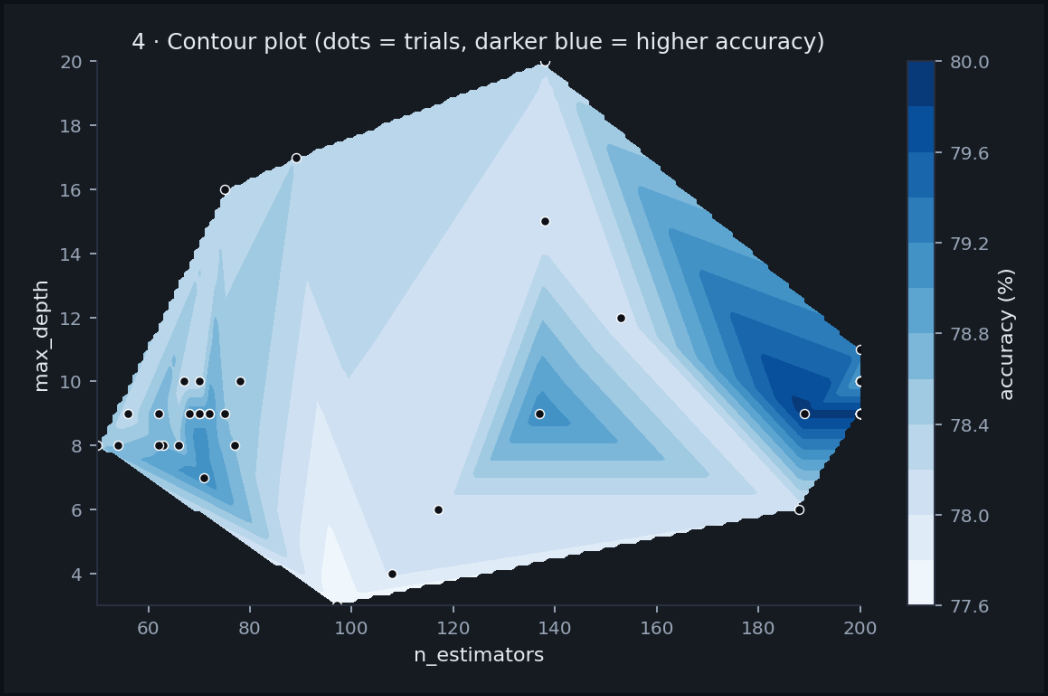


Figure B11 — Contour plot (our reproduction): dots are trials; dense clusters of dots sit on the dark-blue, high-accuracy zones.

5 · Parameter importance plot

Which hyperparameter matters more for the score? In the video max_depth had about **85 %** importance versus **15 %** for n_estimators — on *that* dataset. (In our reproduction the order is the opposite.) Importance depends on the dataset and the ranges, never assume it.

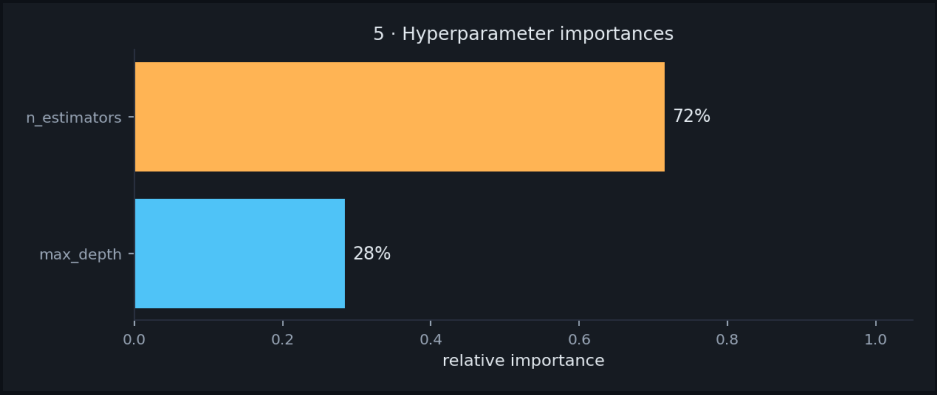


Figure B12 — Importances estimated from our run with a random-forest regressor on the trial table (Optuna uses a related, fANOVA-style method).

> **✔ Why this matters**
>
> A tuning library that only prints a best value is a black box. These plots let you shrink the ranges for a second, finer study, drop unimportant hyperparameters, and decide when to stop.

B8 · Define-by-Run: dynamic search spaces (the lecturer's favourite)

So far we *assumed* Random Forest is the best model. In reality you never know whether Logistic Regression, SVM, Random Forest or XGBoost will win. The ideal workflow runs all of them — but then you must tune each one separately (four tunings for four models), which takes a lot of time. What if you tune **once** and learn *both* which algorithm is best *and* its best hyperparameters?

Optuna solves this with **dynamic search spaces**: make "which algorithm to use" a hyperparameter. Call it `classifier` with options like SVC, RandomForest and GradientBoosting. Then, depending on its value in the current trial, create a *different* search space: SVM-specific hyperparameters if SVC was chosen, Random-Forest ones if RandomForest was chosen, and so on. The search space is **defined while the code runs** — hence "define-by-run". This works because TPE is tree-structured (B3).

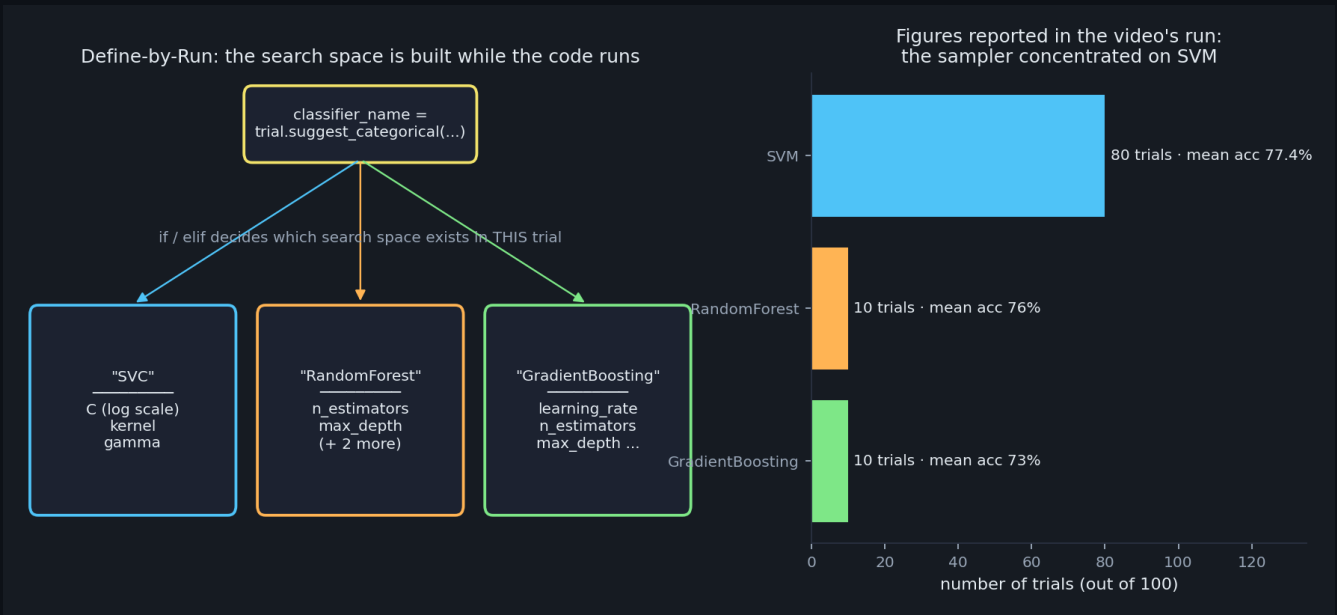


Figure B13 — Left: the conditional search-space tree. Right: the outcome the lecturer reported (see the table below).

define-by-run objective

```
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

def objective(trial):
    classifier_name = trial.suggest_categorical("classifier", ["SVC", "RandomForest", "GradientBoosting"])

    if classifier_name == "SVC":
        c      = trial.suggest_float("svc_c", 1e-2, 1e3, log=True)
        kernel = trial.suggest_categorical("svc_kernel", ["linear", "rbf", "poly"])
        gamma  = trial.suggest_categorical("svc_gamma", ["scale", "auto"])
        model  = SVC(C=c, kernel=kernel, gamma=gamma)

    elif classifier_name == "RandomForest":
        model = RandomForestClassifier(
            n_estimators=trial.suggest_int("rf_n_estimators", 50, 300),
            max_depth=trial.suggest_int("rf_max_depth", 3, 20),
            min_samples_split=trial.suggest_int("rf_min_samples_split", 2, 10),
            min_samples_leaf=trial.suggest_int("rf_min_samples_leaf", 1, 5),
            random_state=42)

    else:  # "GradientBoosting"
        model = GradientBoostingClassifier(
            learning_rate=trial.suggest_float("gb_learning_rate", 1e-3, 0.3, log=True),
            n_estimators=trial.suggest_int("gb_n_estimators", 50, 300),
            max_depth=trial.suggest_int("gb_max_depth", 2, 8),
            random_state=42)

    return cross_val_score(model, X_train, y_train, cv=3, scoring="accuracy").mean()

study = optuna.create_study(direction="maximize")      # TPESampler is the default
study.optimize(objective, n_trials=100)                # more trials: the space is bigger
print(study.best_params)
```

Whatever model is built — SVM, Random Forest or Gradient Boosting — the rest is the same: cross-validate and return the mean accuracy. The if / elif chooses which hyperparameters exist in this trial, and the TPE sampler picks *all* of them (the algorithm *and* its hyperparameters).

Reading the results (the video's run)

Item	Result in the video
Trials run	100
Best algorithm	SVM (SVC) with C ≈ 122, kernel = linear, gamma = scale
Trials spent per algorithm	SVM 80, GradientBoosting 10, RandomForest 10
Mean accuracy per algorithm	SVM 77.4 %, RandomForest 76 %, GradientBoosting 73 %

Interpretation: the sampler noticed early that SVM scored best, so it spent 80 of the 100 trials tuning SVM instead of wasting them on weaker models — Bayesian optimisation allocating effort intelligently. Every trial is available as a DataFrame:

analysing all 100 trials

```
df = study.trials_dataframe()                      # trial number, value, start/end time, duration, params_*
print(df[["number", "value", "datetime_start", "datetime_complete", "duration"]].head())

print(df["params_classifier"].value_counts())      # how often each algorithm was tried
print(df.groupby("params_classifier")["value"].mean())   # average accuracy of each algorithm
```

> **● Why define-by-run is powerful**
> Instead of hyperparameter-tuning one algorithm you optimise any number of algorithms together — in one study, one dashboard, one result. Pre-processing choices (scaler type, imputation strategy, PCA yes/no) can be made hyperparameters in exactly the same way.

B9 · Pro level: features that make Optuna production-ready

Search-space toolbox

Call	Use for
`trial.suggest_int(name, low, high, step=1, log=False)`	Integers (depth, number of trees, layers).
`trial.suggest_float(name, low, high, log=True)`	Real numbers; **log=True** for scales spanning orders of magnitude (learning rate 1e-5…1e-1, regularisation C).
`trial.suggest_categorical(name, choices)`	Choices (algorithm name, kernel, optimiser, activation).

Pipelines: tune pre-processing without leakage

pipeline objective

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer

def objective(trial):
    strategy = trial.suggest_categorical("impute", ["mean", "median"])
    model = make_pipeline(SimpleImputer(strategy=strategy), StandardScaler(),
                          RandomForestClassifier(n_estimators=trial.suggest_int("n", 50, 300),
                                                 max_depth=trial.suggest_int("d", 3, 20), random_state=0))
    return cross_val_score(model, X_train_raw, y_train, cv=5).mean()   # imputer re-fitted in every fold
```

Pruning: stop bad trials early

If your model trains in steps (epochs of a neural network, boosting rounds), you can report intermediate scores. A **pruner** compares them with earlier trials and aborts hopeless runs, saving a huge amount of compute — essential for deep learning.

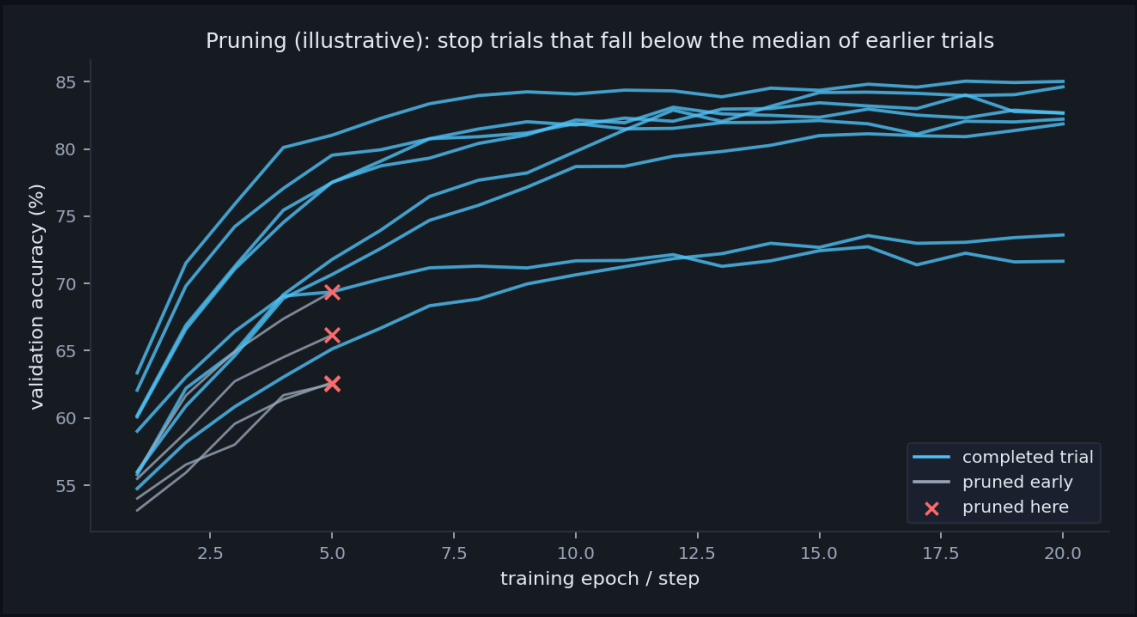


Figure B14 — Pruning (illustrative simulation): trials whose validation curve falls below the median of the earlier trials are stopped (red ×); only promising ones finish.

pruning with MedianPruner

```
from sklearn.linear_model import SGDClassifier

def objective(trial):
    alpha = trial.suggest_float("alpha", 1e-6, 1e-1, log=True)
    clf = SGDClassifier(alpha=alpha, random_state=0)
    for epoch in range(20):
        clf.partial_fit(X_train, y_train, classes=np.unique(y_train))
        acc = clf.score(X_val, y_val)
        trial.report(acc, step=epoch)              # tell Optuna the intermediate score
        if trial.should_prune():                   # the pruner decides
            raise optuna.TrialPruned()
    return acc

study = optuna.create_study(direction="maximize",
                            pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=5))
study.optimize(objective, n_trials=50)
```

Persistent storage, resuming and distributed / parallel computing

The lecture mentions distributed computing: different trials can run on different devices, so the whole study finishes faster. In Optuna you do this by pointing every worker (threads, processes or machines) to the same **storage** database.

storage + parallelism

```
study = optuna.create_study(study_name="rf_tuning", storage="sqlite:///optuna.db",
                            direction="maximize", load_if_exists=True)   # resume if it already exists
study.optimize(objective, n_trials=100, n_jobs=4)     # 4 parallel threads in this process

# More workers (other terminals / machines) simply run the same script pointing at the same storage:
#   they all pull trials from, and push results to, the same study.
# For many users or machines use a server database (PostgreSQL / MySQL) instead of SQLite.
```

Integrations and ecosystem

The lecture notes that Optuna's integration with other libraries is very strong: **scikit-learn, Keras, PyTorch, MLflow** (and more such as LightGBM, XGBoost, PyTorch Lightning, Weights & Biases). Most integrations ship in the separate `optuna-integration` package. For example `OptunaSearchCV` is a drop-in replacement for `RandomizedSearchCV`, and an MLflow callback logs every trial. The `optuna-dashboard` package gives a live web UI over your storage.

OptunaSearchCV

```
# optuna-integration provides sklearn-style search and framework callbacks
from optuna.integration import OptunaSearchCV
from optuna import distributions as d

search = OptunaSearchCV(RandomForestClassifier(random_state=0),
                        {"n_estimators": d.IntDistribution(50, 300),
                         "max_depth":    d.IntDistribution(3, 20)},
                        n_trials=50, cv=3, random_state=0)
search.fit(X_train, y_train); print(search.best_params_)
```

More power tools

multi-objective, enqueue, timeout, callbacks, seeds

```
# Multi-objective: maximise accuracy, minimise model size (returns a Pareto front)
study = optuna.create_study(directions=["maximize", "minimize"])
# objective returns (accuracy, n_parameters); inspect study.best_trials

# Warm-start with settings you already know are good
study.enqueue_trial({"n_estimators": 115, "max_depth": 18})

# Stop by time or with a callback
study.optimize(objective, timeout=600)                              # at most 10 minutes
study.optimize(objective, n_trials=500,
               callbacks=[lambda s, t: s.stop() if s.best_value > 0.95 else None])  # early stop

# Quieter logs, reproducible sampling
optuna.logging.set_verbosity(optuna.logging.WARNING)
sampler = optuna.samplers.TPESampler(seed=42)
```

B10 · Best practices, cheat-sheet and interview questions

Best practices

- **Keep a final test set untouched.** Tune with cross-validation on the training data only; evaluate once on the test set (as in B5).
- **Use log-scale** for learning rates, regularisation and anything spanning orders of magnitude.
- **Start wide, then narrow.** Use the slice / contour / importance plots to shrink ranges and run a second study.
- **Do not over-tune.** If the history plot is flat, stop; use more data or better features instead.
- **Seed everything** (sampler, model, CV splits) if you need reproducibility, and run several seeds before declaring a winner.
- **Make the objective cheap first** (subsample, fewer epochs) and add pruning if it is a learning curve.
- **Treat the algorithm and pre-processing as hyperparameters** with define-by-run when you are unsure what to use.

● Optuna cheat-sheet

`def objective(trial):` suggest → build model → cross-validate → **return score**.

`study = optuna.create_study(direction='maximize'|'minimize', sampler=..., pruner=..., storage=...)`

`study.optimize(objective, n_trials=N, n_jobs=k, timeout=s)` → `study.best_params`, `.best_value`, `.best_trial`, `.trials_dataframe()`

Samplers: TPE (default, Bayesian), Random, Grid, CMA-ES, GP, NSGA-II. **Plots:** history, parallel coordinate, slice, contour, param importances.

Typical interview questions

Question	Answer
Why Optuna over GridSearchCV / RandomizedSearchCV?	It uses Bayesian optimization (TPE): each trial learns from previous ones, so fewer trainings are needed than Grid, and it is less wasteful than Random.
What is a study, a trial, a sampler?	Study = whole optimisation; trial = one evaluation with one set of values; sampler = algorithm that proposes the next values.
What does the objective function return?	A single number (e.g. mean CV accuracy) that Optuna maximises or minimises according to *direction*.
What is TPE?	Tree-structured Parzen Estimator: models densities of good and bad trials and picks values maximising l(x)/g(x).
What is define-by-run?	Building the search space dynamically inside the objective with Python control flow, so conditional / algorithm-dependent hyperparameters are possible.
How does pruning work?	Intermediate scores are reported; a pruner (e.g. median) stops trials that look worse than previous ones.
How to run Optuna in parallel?	Share a storage database and use n_jobs or several processes / machines.
When should direction be minimize?	When the objective returns a loss (cross-entropy, MSE); maximise for accuracy, F1, R^2.

Practice plan

1. Re-run Sections B5 and B6 on your own dataset; compare TPE, Random and Grid with 3 different seeds.
2. Open every plot in B7 and write two sentences about what each one tells you.
3. Build the define-by-run study of B8 with a fourth algorithm (e.g. XGBoost or Logistic Regression).
4. Add a `Pipeline` with imputation and scaling, and a SQLite storage; stop the notebook halfway and resume.
5. Try pruning on a neural network (Keras / PyTorch) with the matching integration.

Closing thought from the lecture. Optuna is probably the best hyperparameter-tuning framework you can know: it works for machine learning and deep learning equally well, and with it in your arsenal you can build complex ML pipelines easily. Pick your favourite dataset and try it.

References and further reading

- Ester, Kriegel, Sander, Xu (1996). *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.* KDD-96 — the original DBSCAN paper.
- Schubert, Sander, Ester, Kriegel, Xu (2017). *DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN.* ACM TODS.
- Ankerst et al. (1999). *OPTICS: Ordering Points To Identify the Clustering Structure.* Campello et al. (2013). *HDBSCAN.*
- scikit-learn documentation: `sklearn.cluster.DBSCAN`, user guide “Clustering”.
- “Visualizing DBSCAN Clustering” — interactive demo recommended in the lecture (search for it by name).
- Akiba, Sano, Yanase, Ohta, Koyama (2019). *Optuna: A Next-generation Hyperparameter Optimization Framework.* KDD 2019 — the paper behind Optuna; documentation at optuna.org.
- Bergstra, Bardenet, Bengio, Kégl (2011). *Algorithms for Hyper-Parameter Optimization.* NeurIPS — introduces TPE. Bergstra & Bengio (2012). *Random Search for Hyper-Parameter Optimization.* JMLR.
- Shahriari et al. (2016). *Taking the Human Out of the Loop: A Review of Bayesian Optimization.* Proc. IEEE.